



PROGRAMMING I



Table of Contents

MODULE 01.....	1
INTRODUCTION TO VISUAL BASIC AND THE .NET FRAMEWORK	1
1.1. INTRODUCTION.....	1
1.2. INTRODUCTION TO VISUAL BASIC AND THE .NET FRAMEWORK	2
1.2.1. What is the .NET Framework?	4
1.2.2. THE .NET FRAMEWORK ARCHITECTURE	4
1.3. VISUAL STUDIO DEVELOPMENT ENVIRONMENT	8
1.3.1. Create a .NET console application using Visual Studio.....	8
1.4. VB.NET SYNTAXES.....	11
1.5. CONTROL STRUCTURES IN VB.....	16
1.6. LOOPING STATEMENTS IN VB.....	20
MODULE 02.....	25
INTRODUCTION TO OBJECT ORIENTED PROGRAMMING	25
2. INTRODUCTION.....	25
2.1. Advantages and Disadvantages of OOP.....	27
2.2. Encapsulation – Data Protection & Security.....	28
2.3. Inheritance – Code Reusability.....	31
2.4. Polymorphism – Flexibility & Extensibility	34
2.5. Abstraction – Simplifying Complexity.....	35
2.6. EXCEPTION.....	37
MODULE 03.....	39
INTRODUCTION TO WINDOWS FORMS APPLICATION.....	39
3. INTRODUCTION.....	39
3.1. Create a picture viewer Windows Forms App (.NET Framework).....	40
3.1.1. Add controls to the picture viewer Windows Forms App (.NET Framework).....	44
3.1.2. Add code to the picture viewer Windows Forms App.....	47
3.2. Create a timed math quiz (.NET Framework).....	49

MODULE 01

INTRODUCTION TO VISUAL BASIC AND THE .NET FRAMEWORK

1.1. INTRODUCTION

Before diving into the in depths of the course, one can ask himself this question! **What the heck is programming all about?** Well, the term **programming** in its simplest form involves **giving instructions to a computer to make it do what we want**, just like how you can instruct someone on how to bake a cake or drive a car. But in this case, rather than speaking to a human, we write instructions using **high-level languages** such as **Python, Java, or VB.NET** which the computer compiles, into a **low-level language (binary language)**.

For further details programming (**or computer programming**) is the process of creating a set of instructions called a **program** that a computer can execute to perform specific tasks. As already mentioned above these instructions are written in programming languages such as **Python, Java, C#, or VB.NET**. At its core, programming is about designing algorithms (**step-by-step procedures**) and implementing them in code.

In general programming involves **Problem-Solving Skills** which is breaking down a problem into smaller, logical steps. **Algorithm Design** which requires creating a clear plan for how the computer should solve the problem. **Coding**, that is writing the plan in a programming language. **Testing & debugging** whereby, we are ensuring the program works correctly and fixing errors. And lastly but not the least **Maintenance**, here we are updating and improving the program over time. Thus, making it a continuous process/endless cycle.

When talking of real-life examples, think of programming as a recipe:

When you follow a cooking recipe, you add ingredients step by step to create a meal. Similarly, when you program a computer, you write step-by-step instructions for it to complete a task.

But now real-life applications of programming include;

- ❖ **Traffic Lights:** Programmers write code to control when traffic lights change, keeping roads safe.
- ❖ **Smartphones:** Every app you use either it being social media platforms, music streaming platforms, or maps is made using programming.
- ❖ **Banks:** Online banking systems rely on programming for secure transactions.
- ❖ **Video Games:** From simple puzzle games to complex multiplayer experiences, programming brings them to life.
- ❖ **AI Assistants (like Copilot, Gemini, Chat-GPT and more!):** Use programming to understand and respond to your questions.

From the above stated examples one can clearly attest the fact that programming is everywhere! It plays a major role in areas such as;

- ❖ **Web Development:** Websites like **YouTube, Google, and Amazon** are built with programming.
- ❖ **Cybersecurity:** Protecting personal data and preventing hackers.

- ❖ **Medical Technology:** Machines such as MRI scanners use programming to analyse health data. *((Magnetic Resonance Imaging) scanners are medical devices that use strong magnetic fields and radio waves to create detailed images of the inside of the human anatomy).*
- ❖ **Self-Driving Cars:** AI-powered cars follow instructions coded by programmers.
- ❖ **Robotics:** Factories use programmed robots to assemble products efficiently.

With all these brought together one can conclude of the importance of programming in this digital age whereby in respect to;

- ❖ **Automation:** It makes life easier by handling repetitive tasks.
- ❖ **Innovation:** Programming drives new technology, from AI assistants to space exploration.
- ❖ **Connectivity:** It connects people across the world through apps, websites, and social media.
- ❖ **Career Opportunities:** Programmers are in high demand, shaping the future of technology.

In short, programming is the language of the digital world without it, computers wouldn't function the way they do today.

1.2. INTRODUCTION TO VISUAL BASIC AND THE .NET FRAMEWORK

VB.NET stands for **Visual Basic. Network Enabled Technologies. Microsoft** released the .NET platform in **2001**, that supports Visual Basic .NET which is an upgrade to the last version of VB programming language. It is a high-level programming language for the Microsoft .NET Framework. It is also possible to run VB.NET on Linux and MAC operating systems.

Visual Basic or VB is a programming language initially released in **1991**. It was the first programming language that directly supported programmable graphical user interface using language-supplied objects. From that time until **2002**, there were 7 other versions released, each version having features that increased the power of the language.

Visual Basic (VB) and the .NET Framework are fundamental technologies for developing a wide range of applications, primarily within the Microsoft ecosystem.

Visual Basic (VB.NET) is a modern, object-oriented programming language developed by **Microsoft**. It is a part of the **.NET Framework**, which provides a platform for developing applications that run on **Windows, Web, and Mobile devices**. VB.NET is widely used for creating Windows applications due to its simplicity, readability, and integration with the rich libraries of **.NET**.

Question 1: What are differences between Visual Basic & VB.NET?

PARAMETERS	VB .NET	VISUAL BASIC
Programming Language Paradigm	VB.NET is an object-oriented programming language.	Visual Basic is Event Driven programming language.
IntelliSense compatibility	VB.NET supports IntelliSense compatibility.	It does not supports IntelliSense compatibility.

Type	It is a compiled typed language	It is an Interpreter based language
Multithreaded	It supports the concept of multithreaded.	It does not support the multithreaded concept.
Type-safe	It is a type-safe language.	It is not a type-safe language.
Changes in Data type	In VB.NET , VARIANT and CURRENCY variable are removed and a DECIMAL variable is introduced.	In Visual Basic VARIANT and CURRENCY, variables are used and a DECIMAL variable is not introduced.
Parameter passing	In VB.NET parameters are passed by a value.	In Visual Basic parameters are passed by reference.
Data Handling	In VB.NET, data is handled using ADO.net	In VB, data is handled using DAO protocol, after that RDO, is followed by ADO.
Background Compilation	In VB.NET background compilation takes place continuously for every valid statement.	Visual Basic is an interpreted language in this each statement is interpreted at a time to be converted into object code, then into source code so there is no background compilation.
Runtime Environment	VB.NET uses the Common Language Runtime environment.	Visual Basic uses the common Visual Basic Runtime environment.
Forms	There is no shape control and line control in VB.NET form.	There is a shape control and line control in Visual Basic form.
Short Circuit Logic	In VB.NET, Short Circuit Logic is available.	In Visual Basic, there is no concept of Short Circuit Logic.
Application Development	In VB.NET we can create different types of applications such as console applications, web applications, windows applications.	Visual Basic is not versatile for creating different types of applications.

Question 2: What are some characteristics of Visual Basic?

- ❖ **A High-Level Programming Language:** VB is designed to be relatively easy to read and write, using syntax that resembles everyday English. This makes it more accessible to beginners compared to lower-level languages.
- ❖ **Object-Oriented:** VB supports the principles of object-oriented programming (OOP), which allows developers to structure code into reusable components called "objects." This promotes modularity, organization, and efficiency.

- ❖ **Event-Driven:** VB applications respond to "events," which are actions performed by the user (like clicking a button) or triggered by the system. The programmer writes code to specify what should happen when these events occur.
- ❖ **.NET-Based:** Modern VB (VB.NET) is built on top of the .NET Framework, which provides a robust environment and a vast library of pre-built components.

1.2.1. What is the .NET Framework?

The **.NET Framework** is a software development platform that provides a comprehensive environment for building, running, and managing applications. It includes:

- ❖ **Common Language Runtime (CLR):** The execution engine that manages memory, security, and error handling.
- ❖ **Base Class Library (BCL):** A collection of reusable classes for handling tasks like file I/O, database access, and networking.
- ❖ **ASP.NET:** A framework for building dynamic web applications.
- ❖ **Windows Forms & WPF:** Tools for building graphical user interfaces (GUIs).
- ❖ **LINQ (Language Integrated Query):** A feature for querying data efficiently.
- ❖ **ADO.NET:** A data-access technology for working with databases.

Question 3: What are some characteristics of the .NET Framework?

- ❖ **A Software Development Platform:** The .NET Framework provides a comprehensive environment for building, deploying, and running applications. It includes a runtime environment (the CLR) and a class library.
- ❖ **Common Language Runtime (CLR):** The CLR is the heart of the .NET Framework. It manages the execution of .NET applications, providing services like memory management, exception handling, and security.
- ❖ **Framework Class Library (FCL):** The FCL is a massive collection of pre-built classes, interfaces, and types that developers can use in their applications. It offers functionality for various tasks, such as:
 - i. **File input/output**
 - ii. **Database connectivity**
 - iii. **Web development**
 - iv. **Graphical user interfaces**

Note: .NET: It is important to note that .NET Framework has evolved to .NET. Modern .NET is cross-platform and open-source. This means that while the original .NET Framework was primarily for Windows, .NET is designed to run on various operating systems, including Windows, macOS, and Linux. It is also open-source, allowing for community contributions and wider adoption.

1.2.2. THE .NET FRAMEWORK ARCHITECTURE

The .NET Framework Architecture is like a structured system that helps developers build applications efficiently. It is designed to provide a consistent programming model for building applications on Windows. It consists of several key components that work together to execute and manage applications efficiently. Think of it as a well-organized factory where different components work together to create digital products.

Question 4: What are the Main Components of .NET Framework Architecture?**i. Common Language Runtime (CLR)**

This is the execution engine of the .NET Framework. It has as main responsibilities that of;

- ❖ Memory management (automatic garbage collection).
- ❖ Thread management (multithreading support).
- ❖ Exception handling (structured error management).
- ❖ Type safety (ensures code doesn't access memory incorrectly).
- ❖ Security (code access security and role-based security).

It allows programs written in different languages (**C#, VB.NET, F#, etc.**) to run on the same runtime.

ii. NET Framework Class Library (FCL)

This is a vast collection of pre-built classes, interfaces, and value types. It provides reusable functionality for:

- ❖ File I/O
- ❖ Database access (ADO.NET)
- ❖ Networking
- ❖ Collections (lists, dictionaries, etc.)
- ❖ XML manipulation
- ❖ User interface development (Windows Forms, WPF, ASP.NET)

It is divided into **Base Class Library (BCL)** for core features and extended libraries for specialized tasks.

iii. Languages

Multiple programming languages can target the .NET Framework:

- ❖ C#
- ❖ VB.NET
- ❖ F#

All compile into Intermediate Language (IL) code, which the CLR executes.

iv. Common Language Specification (CLS)

It defines a set of rules that all .NET languages must follow and ensures interoperability between languages (**e.g., a class written in C# can be used in VB.NET**).

v. Assemblies

Compiled code units in .NET (**usually .dll or .exe files**).

- ❖ Contain metadata about types, versioning, and security.
- ❖ Enable deployment and version control.

vi. Application Models

These are various frameworks built on top of .NET for specific types of applications:

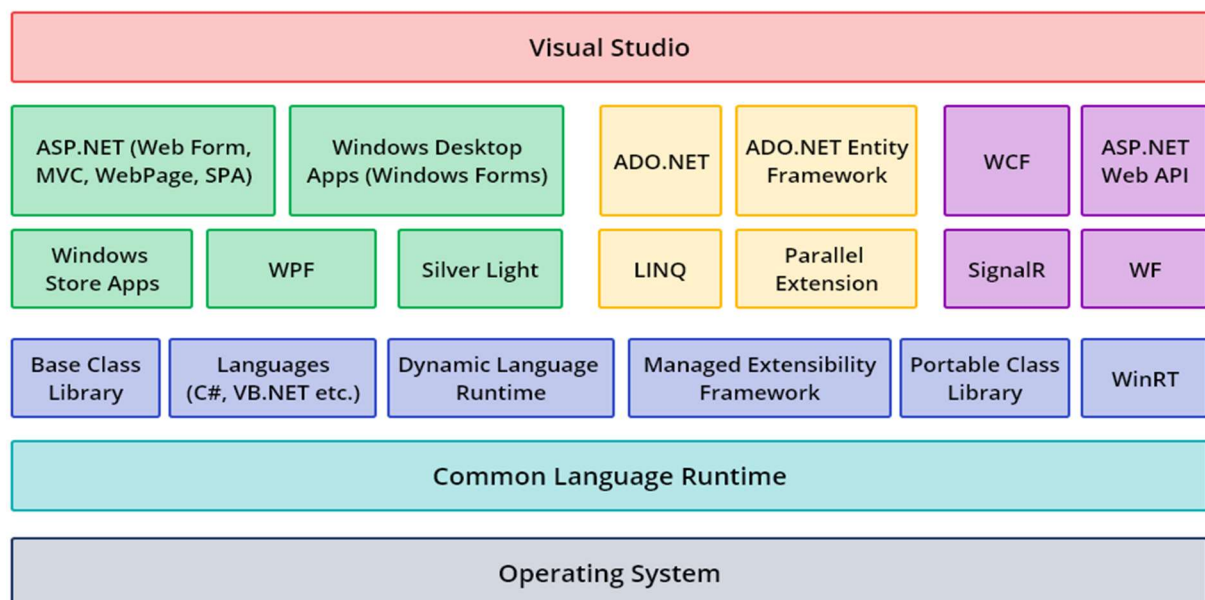
- ❖ ASP.NET → Web applications
- ❖ Windows Forms → Desktop GUI apps
- ❖ WPF (Windows Presentation Foundation) → Rich desktop apps
- ❖ ADO.NET → Data access

In order for us to understand how all these works together, let's get this straight. A developer writes his/her code in **C#, VB.NET, or another .NET language**. From there on the code is compiled into **IL (Intermediate Language)**. The **CLR** executes **IL, handling memory, security, and exceptions**. Application uses **FCL** for built-in functionality. **Assemblies** manage deployment and versioning.

In essence, the .NET Framework Architecture is built around the CLR (runtime) and the FCL (libraries), with languages, assemblies, and application models layered on top to provide a unified development environment.

Below is a diagrammatical representation of the .NET Framework Architecture. Let's break it down with simple explanations and real-life comparisons. As shown in the image below;

.NET FRAMEWORK ARCHITECTURE



- ❖ **Visual Studio – The Workshop:** Visual Studio is the Integrated Development Environment (IDE) where developers write and test code. *As a real-life example imagine Visual Studio as a car workshop where mechanics have all the tools needed to build, fix, and improve vehicles.* In programming, developers use Visual Studio to craft software applications with various built-in tools.

- ❖ **Application Models – Different Types of Products:** This layer represents different types of applications developers can build using .NET. Think of it like a factory producing different products cars, bikes, airplanes, etc.
- ❖ **ASP.NET (Web Applications) - Websites & Online Services:** For example, it operates just like a restaurant website for ordering food online.
- ❖ **Windows Forms (Desktop Applications) → Windows Software:** For example, just like Microsoft Word, which runs on your computer.
- ❖ **WPF (Windows Presentation Foundation) → Rich Graphics Applications:** It's similar to advanced design software like Photoshop, which has visually rich interfaces.
- ❖ **ADO.NET & Entity Framework (Database Management) → Handling large amounts of data:** Take for example just like an online library system that helps users search for books easily.
- ❖ **LINQ (Data Queries) → Retrieving information efficiently:** Same as using Google Search to find specific information instantly.
- ❖ **WCF (Windows Communication Foundation) → Connecting systems:** Same as a phone network that allows communication between different people.
- ❖ **SignalR (Real-time Communication) → Instant messaging:** Similar to a live chat feature on WhatsApp or Messenger.
- ❖ **Base Class Library – The Toolbox:** This is a collection of built-in functionalities like handling files, working with data, and interacting with hardware. Think of it as a toolbox containing essential tools like hammers, screwdrivers, and wrenches needed to build a variety of products.
- ❖ **Programming Languages – The Communication Methods:** Different languages like C#, VB.NET, and F# allow developers to write instructions. Just like we speak English, French, or Spanish, computers understand different programming languages based on how they are written.
- ❖ **Dynamic Language Runtime – The Language Converter:** Allows different programming languages to work together in .NET. Imagine a translator who helps English speakers communicate with French speakers.
- ❖ **Managed Extensibility Framework – Custom Add-ons:** Helps extend applications by adding new features. Like adding new apps to your smartphone to improve functionality.
- ❖ **Portable Class Library – Universal Components:** Allows the same code to run across different platforms (Windows, Web, Mobile). Like a universal charger that works for different phone brands.
- ❖ **WinRT (Windows Runtime) – App Store Integration:** Supports modern Windows apps, especially for the Windows Store. Similar to a system that helps developers create Android or iOS apps for the App Store.
- ❖ **Common Language Runtime (CLR) – The Engine:** Manages the execution of programs and handles memory. Think of it as the engine inside a car that makes everything run smoothly.
- ❖ **Operating System – The Foundation:** The base layer where everything operates. Just like the land where a factory is built —without it, nothing works.

The .NET Framework Architecture works just like a factory, where different departments contribute to creating high-quality products. Understanding each component helps developers create efficient, scalable applications.

Question 5: What is a framework?

In programming, *a framework is a pre-built structure or set of tools that provides a foundation for developing software applications. It includes reusable code, libraries, APIs, and conventions that help streamline development by handling common tasks (like database interaction, security, and user interface design) so developers can focus on writing application-specific logic.*

1.3. VISUAL STUDIO DEVELOPMENT ENVIRONMENT

Visual Studio is Microsoft's Integrated Development Environment (IDE), a powerful tool that provides developers with everything they need to write, create and test .NET applications. As a real-life example, view *Visual Studio as a car workshop where mechanics have all the tools needed to build, fix, and improve vehicles. In programming, developers use Visual Studio to craft software applications with various built-in tools.*

Key Features of Visual Studio

- ❖ **Code Editor:** A sophisticated text editor with features like syntax highlighting, IntelliSense (code completion), and debugging tools.
- ❖ **GUI Designer:** A visual designer for creating graphical user interfaces (GUIs) using drag-and-drop functionality.
- ❖ **Compiler:** A tool that translates VB.NET code into machine-readable code.
- ❖ **Debugger:** A tool for finding and fixing errors in code.
- ❖ **Project Management:** Tools for organizing and managing project files.
- ❖ **Integration with .NET:** Seamless integration with the .NET Framework, providing access to its libraries and features.

Question 6: How Does Visual Studio Work?

- ❖ **Project Creation:** The developer creates a new project in Visual Studio, selecting a project type (e.g., **Windows Forms Application, Console Application**).
- ❖ **Code Writing:** The developer writes VB.NET code in the code editor to define the application's logic.
- ❖ **GUI Design (Optional):** For applications with a user interface, the developer uses the GUI designer to create forms, buttons, and other controls.
- ❖ **Compilation:** Visual Studio uses the VB.NET compiler to translate the code into an intermediate language (IL).
- ❖ **Execution:** When the application is run, the CLR takes the IL code and compiles it into machine code that the computer's processor can execute.

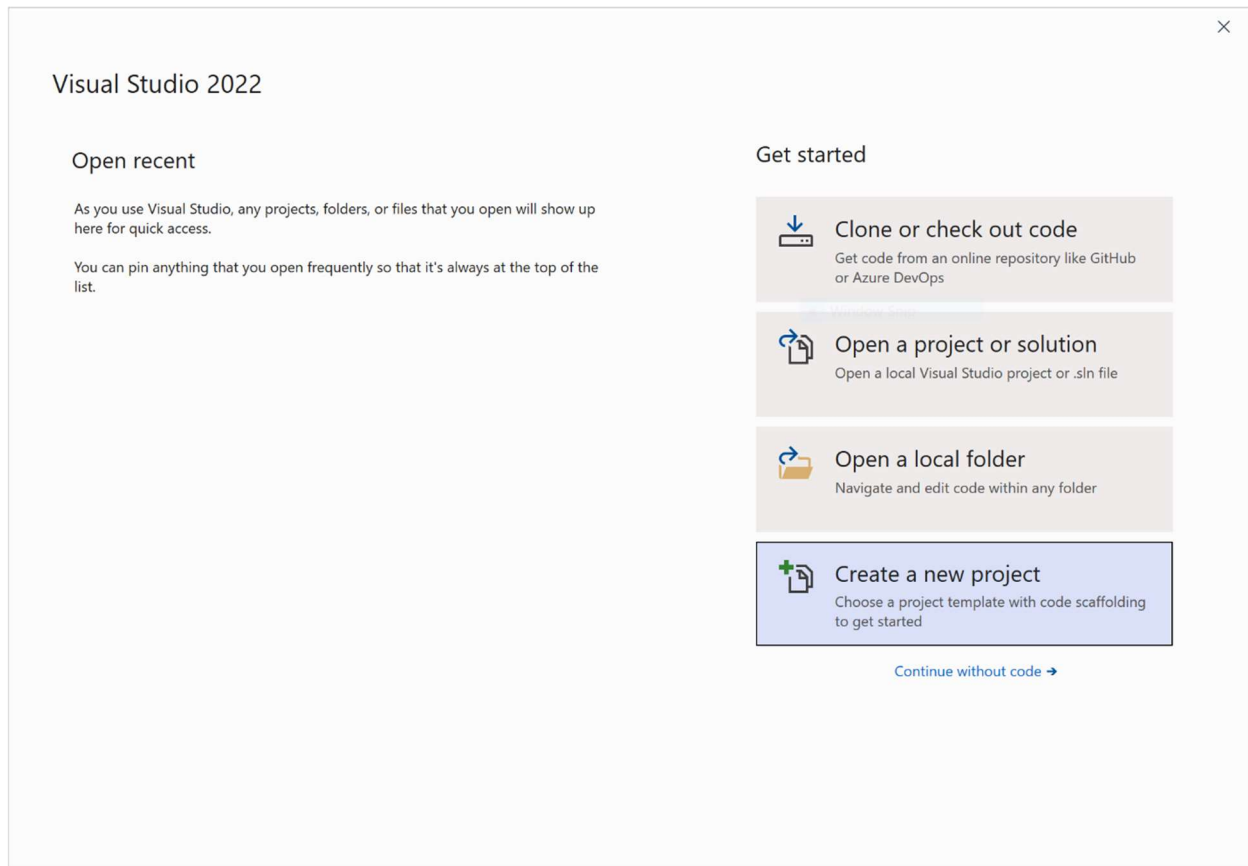
1.3.1. Create a .NET console application using Visual Studio

Create the app

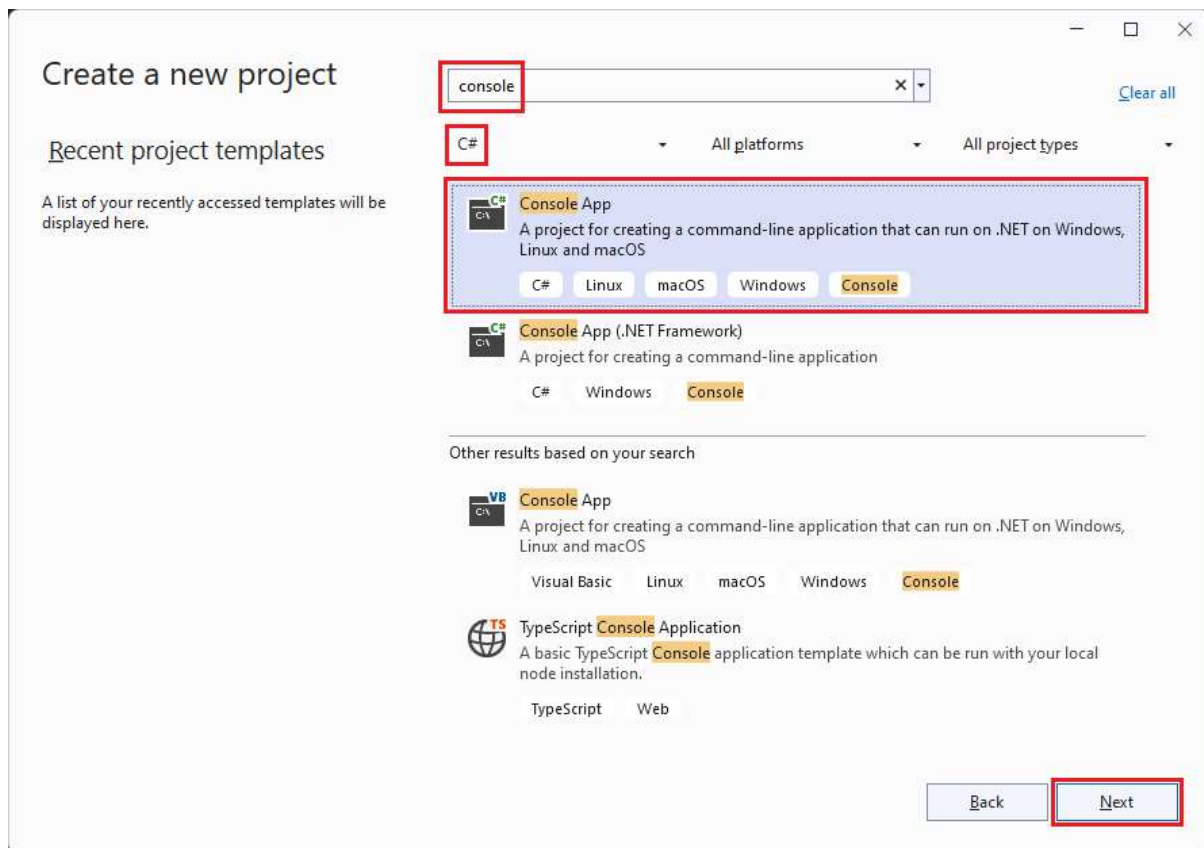
Create a .NET console app project named **"Software Projects"**.

- ❖ Start Visual Studio 2022.

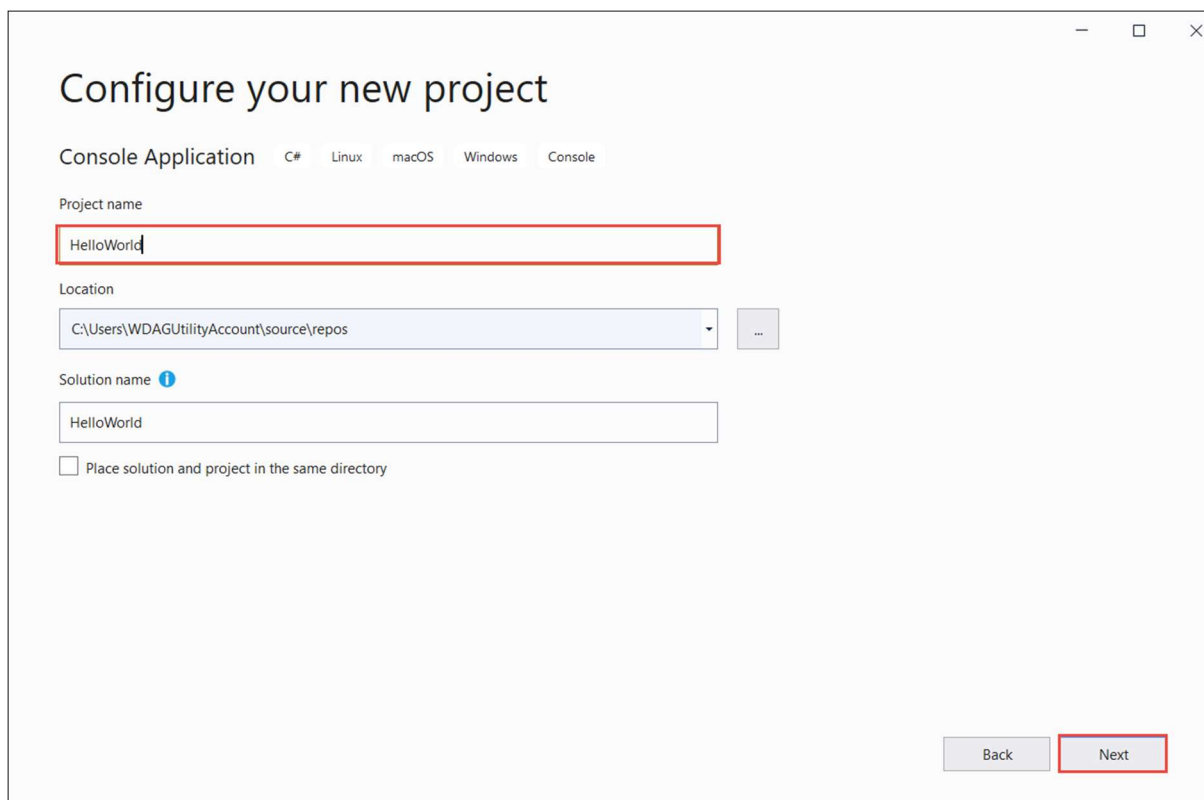
- ❖ On the start page, choose Create a new project



- ❖ On the Create a new project page, enter console in the search box. Next, choose C# or Visual Basic from the language list, and then choose All platforms from the platform list. Choose the Console App template, and then choose Next.



- ❖ In the Configure your new project dialog, enter HelloWorld in the Project name box. Then choose Next.



In the Additional information dialog:

- ❖ Select .NET 8.
- ❖ Select Do not use top-level statements.
- ❖ Select Create.

The template creates a simple application that displays "Hello, World!" in the console window.

```
Namespace HelloWorld  
  
    Friend Module Program  
        Sub Main()  
            Console.WriteLine("Hi Family!")  
        End Sub  
    End Module  
  
End Namespace
```

1.4. VB.NET SYNTAXES

i. Core Structure of a VB.NET Program

A typical VB.NET program is organized into a hierarchical structure

Namespaces (Optional but Recommended):

- ❖ Namespaces help organize your code and prevent naming conflicts, especially in larger projects. They act like containers for classes, modules, and other namespaces.

Syntax:

```
Namespace YourNamespaceName  
    ' Your code (classes, modules, etc.) goes here  
End Namespace
```

Modules or Classes:

- ❖ Modules are primarily used to hold shared procedures (Subroutines and Functions) and data that don't necessarily belong to a specific object. Modules cannot be instantiated as objects.

Syntax:

```
Module YourModuleName  
    ' Procedures and variables go here  
End Module
```

A module is a container for code, similar to a class, but with some differences (modules can't be instantiated as objects). In VB.NET, a simple program often starts with a Module.

Sub Main:

- ❖ The Main subroutine is the entry point of your program. This is where the execution begins. Every VB.NET executable program must have a Main subroutine. The **Sub** keyword is used to define a subroutine. **Main** is usually declared within a module.
- ❖ The parentheses () after Main indicate that it's a subroutine. It might optionally take arguments, but in the simplest "**Hello World**" example, it does not.

Code to Print "Hello World":

- ❖ The Console class, part of the System namespace, provides methods for interacting with the console (command prompt). The **WriteLine ()** method of the Console class writes a line of text to the console.

ii. Variables in VB.NET

In VB.NET, variables are used to store data that your program works with. Think of them as containers that hold values. But before one can use a variable, it is always good to remember that in order for it to be used in our program you need to declare it first, which involves:

- ❖ **Giving it a name:** This is how you'll refer to the variable in your code.
- ❖ **Specifying its data type:** This tells VB.NET what kind of data the variable will hold (e.g., text, numbers, dates). As shown below;

```
Dim variableName As DataType
```

- ❖ **Dim:** This keyword stands for "Dimension" and is used to declare a variable.
- ❖ **variableName:** This is the identifier you choose for your variable. It should be descriptive and follow VB.NET naming conventions (e.g., start with a letter, no spaces).
- ❖ **As:** This keyword separates the variable name from its data type.
- ❖ **DataType:** This specifies the kind of data the variable will store. Common data types include *Integer (whole numbers)*, *Double (floating-point numbers)*, *String (text)*, *Boolean (True or False)*, and *Date (date and time values)*.

```
Dim age As Integer  
Dim name As String  
Dim pi As Double
```

iii. Assigning Values to Variables

Once you've declared a variable, you can assign a value to it using the assignment operator (=):

Let's look at some examples of declaring variables, assigning values, and using them in a VB.NET program:

Example 1: Using variables to store a name and greeting.

Module GreetingModule**Sub Main()***' Declare variables**Dim userName As String**Dim greeting As String**' Assign values to the variables**userName = "Alice"**greeting = "Hello, " & userName & "!" 'The & operator is used for concatenation**' Print the greeting to the console**Console.WriteLine(greeting)**'change the value of the variable**userName = "Bob"**greeting = "Hello, " & userName & "!"**Console.WriteLine(greeting)**End Sub**End Module*

Exercise 1: With the use of concatenation display on the console the statement **“Hi am called Parker, and am 17 years of age, with a height of 1.56m”**.

Exercise 2: With the use of concatenation display on the console the statement **“Hi am called Lichar, and am 17 years of age, with a height of 1.47m and am a female”**.

Example 2: Using variables to perform a calculation

Module CalculationModule**Sub Main()***' Declare variables**Dim num1 As Integer**Dim num2 As Integer**Dim sum As Integer**' Assign values to the variables**num1 = 10**num2 = 20**' Calculate the sum**sum = num1 + num2**' Print the result**Console.WriteLine("The sum of " & num1 & " and " & num2 & " is: " & sum)**End Sub**End Module*

Exercise 3: Write a VB program that performs all 4 arithmetic operations (add, subtract, multiply and divides) with user input, whereby at the level of division an exception needs to be caught whereby *division by 0 is not allowed*.

iv. User-Input

In VB, when dealing with programs that require user-input (that is data entered by the user) we use;

- ❖ **Console.Write()** to display a prompt to the user, asking for input.
- ❖ **Console.ReadLine()** to read the user's input from the console. This input is typically read as a string.

If the input needs to be a data type other than a string (e.g., **Integer**, **Double**), you must convert it using functions like **Integer.Parse()**, **Double.Parse()**, or **TryParse()**. It's good practice to use **TryParse()** to handle potential errors if the user enters invalid input.

Example 3: Request a user to enter his name and age

```
Console.Write("Enter your name: ")
userName = Console.ReadLine()

Console.Write("Enter your age: ")
Dim ageInput As String = Console.ReadLine()
If Integer.TryParse(ageInput, age) Then
    'age is now a valid integer
Else
    Console.WriteLine("Invalid input.")
    'Handles an error
End If
```

Explanation

Console.Write("Enter your age: ")

- ❖ This line displays a prompt on the console, asking the user to enter their age. Console.Write is used so that the cursor remains on the same line, allowing the user to type their input immediately after the prompt.

Dim ageInput As String = Console.ReadLine()

This line does two things:

- ❖ **Dim ageInput As String:** It declares a variable named ageInput of type String. This variable will store the text that the user types in.

Console.ReadLine(): It reads the input that the user enters in the console (until the user presses the Enter key) and assigns that input to the ageInput variable. Critically, Console.ReadLine() always returns a string.

If Integer.TryParse(ageInput, age) Then

- ❖ This line is the most important part for robust user input handling. It attempts to convert the string in `ageInput` into an integer.

Integer.TryParse(): This is a method specifically designed for safely converting strings to integers.

It takes two arguments:

- ❖ **ageInput:** The string you want to convert (the user's input).
- ❖ **age:** An integer variable where the result of the conversion will be stored if the conversion is successful. Note that the `age` variable must be declared before this `If` statement.

It returns a Boolean value:

- ❖ **True:** If the conversion was successful. The `age` variable now contains the integer value of the user's input.
- ❖ **False:** If the conversion failed (e.g., the user entered "abc" instead of a number). The `age` variable will not be changed.

The If statement checks the result of Integer.TryParse():

- ❖ **If True:** The code inside the `Then` block is executed.
- ❖ **If False:** The code inside the `Else` block is executed.

'age is now a valid integer

This is a comment. It's not executed by the computer; it's just there to explain to programmers what's happening. If the `Integer.TryParse()` method returns `True`, then the variable `age` now holds the integer value entered by the user.

Console.WriteLine("Invalid age entered.")

This line is inside the `Else` block. It's executed only if `Integer.TryParse()` returns `False`, meaning the user entered invalid input. It displays an error message to the user.

'Handle error

This is another comment, reminding you (**that is the programmer**) that you should add this code to handle the error. In a real program, you might:

- ❖ Ask the user to enter the age again.
- ❖ Set the age to a default value.
- ❖ Stop the program.

Note: There is a huge difference between the `Console.Write()` and the `Console.WriteLine()` whereby;

- ❖ **Console.Write():** outputs text to the console without inserting a new line
- ❖ **Console.WriteLine():** outputs text and later on inserts a new line

In summary, this code safely gets the user's age, checks if the input is a valid number, and either stores the number in the age variable or displays an error message. It's a good example of robust input validation.

1.5. CONTROL STRUCTURES IN VB

Control structures are the fundamental building blocks that dictate the flow of execution of your code. Think of them as the traffic signals and road layouts that guide the computer through the instructions you've written. Without them, your program would simply execute line by line, from top to bottom, without any ability to make decisions or repeat actions.

Control structures allows us to create programs that are:

- ❖ **Decision-making:** They enable your program to choose different paths of execution based on whether certain conditions are true or false (*like the If...Then...Else and Select Case statements we discussed*).
- ❖ **Repetitive:** They allow you to execute a block of code multiple times until a specific condition is met or for a predetermined number of iterations (*like the various For and Do loops we covered*).
- ❖ **Organized and Readable:** They help structure your code logically, making it easier to understand, maintain, and debug.

Essentially, control structures provide the means to deviate from the simple sequential execution of code. They give you the power to control which statements are executed and when and how many times they are executed.

The conditional statements (*If...Then...Else, Select Case*) and looping statements (*For...Next, For Each...Next, Do While...Loop, Do...Loop While, Do Until...Loop, Do...Loop Until*) that we've already discussed are the primary types of control structures in Visual Basic.

Let's explore the main types of conditional statements in VB with some illustrative examples:

i. If Else Statement

This is the most fundamental conditional statement. It allows you to execute a block of code if a condition is true, and optionally execute another block of code if the condition is false.

Example 4: An If Else Statement to check whether you are eligible to vote or not

```
Dim age As Integer = 25

If age >= 18 Then
    Console.WriteLine("You are eligible to vote.")
Else
    Console.WriteLine("You are not yet eligible to vote.")
End If
```

Exercise 3: Convert the VB program above to one with user input, whereby when a user enters 18 and above it displays “*You are eligible to vote*”, 17 and below displays “*You are not eligible to vote*” now when a user enters a character other than a number which could be

either a **letter** such as *k* or a **special character** such as an # or a ? it displays *"Sorry invalid input, only numbers are required"*.

- ❖ **If age >= 18 Then:** This is the condition being evaluated. If the value of the age variable is greater than or equal to 18, the code following Then will be executed.
- ❖ **Console.WriteLine("You are eligible to vote."):** This code is executed if the condition is true.
- ❖ **Else:** This keyword introduces the block of code that will be executed if the initial condition is false.
- ❖ **Console.WriteLine("You are not yet eligible to vote."):** This code is executed if the condition is false.
- ❖ **End If:** This marks the end of the If...Then...Else block.

You can also have an If..Then statement without the Else part if you only want to execute code when a condition is true:

Example 5: An If statement associated with a Boolean datatype

```
Dim isRaining As Boolean = True

If isRaining Then
    Console.WriteLine("Remember to take your umbrella!")
End If
```

Exercise 4: Write a program using VB to check if the temperature is greater than 25. If the condition is true, the message box **"It's a warm day!"** will be displayed. If the condition is false, nothing within the If block will be executed.

Exercise 5: Using an If Else statement write a program in VB to check the value of the **isMember** boolean variable. **If isMember is True**, the first message box is shown. Otherwise (**if isMember is False**), the message box in the Else block is displayed.

ii. If ElseIf Else Statement

This allows you to check multiple conditions in sequence. If the first condition is false, it moves to the next Else If condition, and so on. The optional Else block at the end is executed if none of the preceding conditions are true.

```
Dim grade As Integer = 85

If grade >= 90 Then
    Console.WriteLine("Excellent!")
ElseIf grade >= 80 Then
    Console.WriteLine("Very Good!")
ElseIf grade >= 70 Then
    Console.WriteLine("Good.")
ElseIf grade >= 60 Then
    Console.WriteLine("Satisfactory.")
ElseIf grade >= 50 Then
    Console.WriteLine("Fair.")
```

```
Else  
    Console.WriteLine("Needs Improvement.")  
End If
```

Exercise 6: Convert the above program above into one with user-input whereby based on the marks entered by the user he/she can earn grades such as **A, B, C, D, E or F** below marks of **50** gives an **F** mark above **90** gives **100** but the user can't enter a number greater than **100** neither can he enter a non-numerical value, if he does it must return an **invalid message**

Exercise 7: Develop a VB.NET **Payroll Application** that captures an **employee's name, hours worked, and hourly rate**, ensuring all numerical inputs are validated as positive values; the program must calculate gross pay using a **1.5 * overtime multiplier** for any hours exceeding **40**, deduct a **flat 20% tax**, and display a final summary formatted in **XAF**.

The conditions are evaluated in order. Once a condition is found to be true, the corresponding code block is executed, and the rest of the Else If and Else blocks are skipped.

iii. Nested If Statements

You can place If statements inside other If statements to create more complex decision-making structures.

```
Dim hasLicense As Boolean = True  
Dim age As Integer = 17  
  
If hasLicense Then  
    If age >= 18 Then  
        Console.WriteLine("You are eligible to drive.")  
    Else  
        Console.WriteLine("You have a license but are not yet old enough to drive  
independently.")  
    End If  
Else  
    Console.WriteLine("You need a driver's license to drive.")  
End If
```

Here, the outer If checks if the person has a license. Only if they do, the inner If statement checks their age. Think of it like a security clearance whereby first, you check if a student is allowed in the building (**Outer If**), then you check if they are allowed in the classroom (**Inner If**).

Exercise 8: Develop a VB.NET **University Checkpoint** whereby the "**Outer If**" acts as the campus gatekeeper, whereby you must present your **School Fees Receipt** which is required to enter campus and the "**Inner If**" acts as the exam room invigilator whereby you must present to him your **NIC** which is required to enter the exam room. You cannot reach the exam room if you haven't even made it onto the campus!

Exercise 9: Develop a VB.NET **Cinema Ticket Calculator** that determines the price of a movie ticket based on the **Age** and **Membership Status**, whereby the program will ask for

the user's age. If the user is under **18** that is him being a minor, it now checks if they have an ID. If yes, the price is **3.000 XAF**. If no, it's **8.000 XAF**. But if the user is **18** or older that is him being an adult, it checks if the user has a **VIP Membership**. If yes, the price is **12.000 XAF** If no, it's **15.000 XAF**. [Here, the outer If checks if the person has a license. Only if they do, the inner If statement checks their age.](#)

iv. **Select Case Statement:**

The **Select Case** statement provides a more structured way to handle multiple conditions based on the value of a single expression. It can be cleaner and easier to read than a long chain of Else If statements when you're comparing one variable against several possible values.

```
Dim dayOfWeek As String = "Wednesday"
```

```
Select Case dayOfWeek
```

```
Case "Monday"
```

```
Console.WriteLine("Start of the work week.")
```

```
Case "Tuesday"
```

```
Case "Wednesday"
```

```
Case "Thursday"
```

```
Console.WriteLine("Mid-week.")
```

```
Case "Friday"
```

```
Console.WriteLine("Almost the weekend!")
```

```
Case "Saturday"
```

```
Case "Sunday"
```

```
Console.WriteLine("Weekend!")
```

```
Case Else
```

```
Console.WriteLine("Invalid day.")
```

```
End Select
```

- ❖ **Select Case dayOfWeek:** This specifies the expression whose value will be compared against the Case values.
- ❖ **Case "Monday":** This checks if the value of **dayOfWeek** is "Monday".

You can also use ranges with the **to** keyword (e.g., Case 1 To 5).

The Case Else block is executed if the expression's value doesn't match any of the preceding Case values.

- ❖ **End Select:** This marks the end of the Select Case block.

While **If...ElseIf** is great for ranges (**like grades**), the **Select Case** statement is the king of **"Menu Selection."** It's cleaner, easier to read, and perfect for when you have a specific list of options.

Exercise 10: Develop a VB.NET **Beverage Menu Selector** which display a list of **4 items (Coffee, Tea, Beer, Water)**. From there on the program will ask the user to enter the **Number** of their choice (**1-4**). Use **Select Case** to display the price of that specific item. And the

program must be capable of handling "Invalid" choices (like if they type 99) using Case Else.

These conditional statements are fundamental tools in our VB toolkit. They empower our programs to respond dynamically to different situations and user inputs, making them much more versatile and intelligent!

1.6. LOOPING STATEMENTS IN VB

Looping statements in Visual Basic are our go-to tools when we need to execute a block of code repeatedly. Instead of writing the same lines of code multiple times, we can use a loop to automate the process. *Think of them as tireless workers that perform a task until a specific condition is met or for a predetermined number of times.*

Loops are the "engine" of any program, allowing you to repeat tasks without writing the same code a thousand times

Let's dive into the different types of looping statements available in VB with clear examples:

i. For...Next Loop

This loop is ideal when you know in advance how many times you need to execute a block of code. It uses a counter variable that increments or decrements through a specified range.

```
' Example 1: Counting from 1 to 5
For i As Integer = 1 To 5
    Console.WriteLine("The current value of i is: " & i)
Next i

' Example 2: Counting down from 10 to 1 with a step of -2
For j As Integer = 10 To 2 Step -2
    Console.WriteLine ("The current value of j is: " & j)
Next j
```

- ❖ **For i As Integer = 1 To 5:** This initializes a counter variable i as an integer and sets its starting value to 1 and its ending value to 5.
- ❖ **Console.WriteLine("The current value of i is: " & i):** This is the block of code that will be executed in each iteration of the loop.
- ❖ **Next i:** This increments the counter variable i by the default step of 1 and returns to the beginning of the loop if the counter is still less than or equal to the ending value.
- ❖ **Step -2:** In the second example, this specifies that the counter variable j should be decremented by 2 in each iteration.

Exercise 11: Develop a VB.NET program that ask the user for a number (e.g., 5) and display its **multiplication table from 1 to 12.**

ii. For Each...Next Loop:

This loop is specifically designed to iterate over the elements of a collection (like an array or a list). It executes the loop body once for each element in the collection.

```
' Example: Iterating through an array of names
Dim names() As String = {"Merline", "Omina", "Laurand"}

For Each name As String In names
    Console.WriteLine(" ("Hello, " & name & "!")
Next name
```

- ❖ **For Each name As String In names:** This declares a variable name of type String, which will hold the value of each element in the names array during each iteration.
- ❖ **names:** This is the collection you want to iterate over.

In each iteration, the name variable automatically takes on the value of the next element in the names array.

iii. Do While...Loop:

This loop continues to execute its block of code as long as a specified condition is true. The condition is checked before each iteration. If the condition is false from the start, the loop body might not execute at all.

```
Module Negative
Sub Main()
    Dim number As Integer

    Do
        Console.Write("Enter a number (negative to exit): ")
        Try
            number = Convert.ToInt32(Console.ReadLine())

            If number >= 0 Then
                Console.WriteLine("You entered: " & number)
            End If
        Catch ex As Exception
            Console.WriteLine("Invalid input! Please enter a valid number.")
            Continue Do ' Restart the loop when an invalid input is detected
        End Try
    Loop While number >= 0

    Console.WriteLine("Loop exited. Goodbye!")
End Sub
End Module
```

This program asks the user to enter a number and continues looping until the user enters a negative number. The code inside the Do While and Loop will be executed repeatedly as long as the condition remains true.

Explanation:

- ❖ The loop starts and prompts the user to enter a number.
- ❖ If the number is negative, the loop stops.

- ❖ If the number is zero or positive, it displays the entered value and continues looping.
 - ❖ **Console.ReadLine():** This reads input from the user as a string. When the user types a number and presses Enter, it captures that input.
 - ❖ **Convert.ToInt32(Console.ReadLine()):** Since Console.ReadLine() returns a string, we use Convert.ToInt32() to convert the input into an integer. This ensures we can perform numerical operations on it.
 - ❖ **number = Convert.ToInt32(Console.ReadLine()):** This assigns the converted integer to the variable **number**, allowing us to use it in conditions or calculations.
 - ❖ **Try-Catch Block:** Catches any errors if the user enters a non-numeric value.
 - ❖ **Error Message:** Notifies the user that they entered an invalid input.
 - ❖ **Continue Do Statement:** If an error occurs, the loop restarts without exiting.
- Exercise 12:** Develop a VB.NET program that keeps asking the user for a password until they type the correct password being "**Sonolivefest**". With the use of **the Do...Loop Until**.

iv. Do...Loop While

This loop is very similar to Do While...Loop, but the condition is checked after each iteration. This means that the code inside the loop will always execute at least once, even if the condition is false from the beginning.

```
Module Do
    Sub Main()
        Dim counter As Integer = 1
        ' Start the Do loop
        Do
            Console.WriteLine("Counter value: " & counter)
            counter += 1 ' Increment counter
        Loop Until counter > 5

        Console.WriteLine("Loop finished!")
    End Sub
End Module
```

Explanation

- ❖ The loop begins with counter = 1.
- ❖ It prints the current value of counter.
- ❖ The counter increments by 1 in each iteration.
- ❖ The loop runs until **counter > 5**, meaning it stops when counter reaches 6.

v. Do Until...Loop:

This loop continues to execute its block of code as long as a specified condition is false. The condition is checked before each iteration. The loop stops when the condition becomes true.

```
Module DoUntil
    Sub Main()
        Dim counter As Integer = 1
```

```
' Start the Do Until loop  
Do Until counter > 5  
    Console.WriteLine("Counter value: " & counter)  
    counter += 1 ' Increment counter  
Loop  
    Console.WriteLine("Loop finished!")  
End Sub  
End Module
```

Explanation

- ❖ The loop starts with counter = 1.
- ❖ It prints the value of counter in each iteration.
- ❖ The counter increases by 1 each time.
- ❖ The loop runs until counter > 5, meaning it stops once counter reaches 6.

vi. Do...Loop Until:

Similar to Do Until...Loop, but the condition is checked after each iteration. This ensures that the loop body executes at least once. This program starts with counter = 1 and keeps looping until the **counter reaches 5**.

```
Module DoUntil  
    Sub Main()  
        Dim counter As Integer = 1  
  
        ' Start the Do Until loop  
        Do Until counter > 5  
            Console.WriteLine("Counter value: " & counter)  
            counter += 1 ' Increment counter  
        Loop  
  
        Console.WriteLine("Loop finished!")  
    End Sub  
End Module
```

Explanation

- ❖ The loop starts with counter = 1.
- ❖ It prints the value of counter in each iteration.
- ❖ The counter increases by 1 each time.
- ❖ The loop runs until **counter > 5**, meaning it stops once counter reaches 6.

Note: A Do While loop continues to execute as long as the specified condition remains True. It stops when the condition becomes False. Meanwhile, A Do Until loop continues to execute as long as the specified condition remains False. It stops when the condition becomes True.

Exercise 13: Write a VB.NET **Countdown Timer** with the use of the **While...End While Loop**, which checks the condition before entering the loop. If the starting number is already **0**, it won't run at all. **It starts at 10 and count down to 1, then print "Blast off!"**

Looping statements are powerful tools for automating repetitive tasks and working with collections of data in your Visual Basic applications. Choosing the right type of loop depends on whether you know the number of iterations beforehand, whether you need to iterate over a collection, or whether you need to repeat an action based on a specific condition.

MODULE 02

INTRODUCTION TO OBJECT ORIENTED PROGRAMMING

2. INTRODUCTION

Object-Oriented Programming (OOP) is a fundamental **programming paradigm** that revolves around the concept of "**objects**." These objects are instances of classes and can contain both data (in the form of fields, attributes, or properties) and code (in the form of procedures or methods) that manipulates this data. OOP aims to model real-world entities and their interactions within software, leading to more modular, reusable, and maintainable code.

Question 6: What is a programming paradigm?

Programming paradigms refers to the **different styles or approaches programmers use to structure and solve problems in code**. Each paradigm influences how we (**programmers**) think about problems and how we design solutions.

There exist a wide variety of different programming paradigms, but at our current level we shall limit ourselves to just 2 for the meantime, that is;

- ❖ **Procedural Programming:** Organizes our code into reusable procedures or functions. As features it builds on imperative programming and emphasizes on modularity and reusability. Examples here include; **C, Pascal, early versions of BASIC**.
- ❖ **Object-Oriented Programming (OOP):** It models real-world entities as objects with attributes and behaviours. As features it includes the 4 main pillars/concepts which are **Encapsulation, Inheritance, Polymorphism & Abstraction**. And promotes code reusability and scalability. Examples here include; **Java, C#, VB.NET, Python (OOP style)**.

At its core, OOP is built upon the principles of creating **classes** and **objects**.

i. Classes

A **class is like a blueprint or template for creating objects**. It defines the properties (attributes) and methods (functions) that the objects will have. Think of it as a mold that dictates the structure and behaviour of things created from it.

Example 1: The class Car followed by its properties

```
Public Class Car  
' Properties  
Public Brand As String  
Public Model As String  
  
' Method  
Public Sub DisplayInfo()  
    Console.WriteLine("Car: " & Brand & " " & Model)  
End Sub  
End Class
```

ii. Objects

An object is an instance of a class. When you create an object, you are making a real version of the class, complete with its properties.

Example 2: The object to access the properties of the class Car

```
Dim myCar As New Car()  
myCar.Brand = "Toyota"  
myCar.Model = "Corolla"  
myCar.DisplayInfo() ' Output: Car: Toyota Corolla
```

But wait a minute.....What the heck is that **public** keyword used for? Well, the public keyword used in our program is one of the **3 Access Modifiers**. Well one can still ask himself what equally is an access modifier?

iii. Access Modifiers

In Object-Oriented Programming (OOP), access modifiers control the visibility and accessibility of class members (properties). They help enforce **encapsulation**, ensuring that data is protected and only accessed where necessary.

Types of Access Modifiers

- ❖ **Public:** Can be accessed from anywhere within the program.
- ❖ **Private:** Can be accessed only within the same class.
- ❖ **Protected:** Can be accessed within the same class and its derived (child) classes.

As a real-life analogy think of it as a house with different rooms and doors having specific access restrictions.

- ❖ **Public:** The main entrance door—anyone can enter.
- ❖ **Private:** Your bedroom—only you can enter.
- ❖ **Protected:** A family room—only family members (inherited classes) can access.

Example 3: The Private Access Modifier

```
Public Class BankAccount  
    Private Balance As Decimal = 1000  
  
    Public Sub ShowBalance()  
        Console.WriteLine("Your balance is: " & Balance)  
    End Sub  
End Class  
  
Dim myAccount As New BankAccount()  
myAccount.ShowBalance() ' Works, but Balance itself is private
```

The Balance variable is **private**, so it cannot be accessed directly outside the class.

Example 4: The Protected Access Modifier

```
Public Class Animal  
  Protected Name As String  
End Class  
  
Public Class Dog  
  Inherits Animal  
  
  Public Sub SetName(newName As String)  
    Name = newName ' Allowed because Dog inherits Animal  
  End Sub  
End Class
```

The **Name** property is **protected**, meaning it can be accessed in the **Dog** class because **Dog** inherits **Animal**.

Exercise 14: Create a class that represents a **Smartphone**. Each smartphone object should have a **brand** and a **model**, and a method to "**Power On**." Therefore, the **Class** created is **Smartphone**, it has as **Properties** both **Brand**, **Model** and a **Method** called **StartPhone()** Now create 2 different **Objects** (e.g., **iPhone** and **Samsung**) from the same class.

2.1. Advantages and Disadvantages of OOP

Object-Oriented Programming (OOP) has transformed software development by making programs more modular, reusable, and scalable. However, like any programming paradigm, it has its pros and cons.

Advantages of OOP

- ❖ **Encapsulation (Data Security & Privacy)** OOP allows data hiding by restricting access using access modifiers (Private, Protected, etc.). **For example**, a **BankAccount** class can have a Private Balance property to prevent unauthorized modification.
- ❖ **Reusability through Inheritance** Classes can inherit properties and behaviours from other classes, reducing duplicate code. **For example:** A Vehicle class can be inherited by Car and Bike classes, preventing redundant code.
- ❖ **Improved Maintainability & Scalability** Changes to a class are automatically reflected in all objects, making updates easier. Large systems can be built by organizing independent classes.
- ❖ **Abstraction (Simplified Complexity)** Classes can expose only relevant details, hiding complex logic. **For example:** A **PaymentProcessor** class can have a simple MakePayment() method while handling complex banking transactions internally.
- ❖ **Polymorphism (Flexible Code)** Allows the same method name to behave differently in different classes. For example, **DrawShape()** can be defined differently in Circle, Square, and Triangle classes.

Disadvantages of OOP

- ❖ **Performance Overhead** Due to abstraction and encapsulation, OOP-based programs may consume more memory and processing power compared to procedural programming.
- ❖ **Complexity in Small Projects** OOP introduces concepts like classes, objects, inheritance, and polymorphism, which can be unnecessary for simple applications.
- ❖ **Longer Development Time** Structuring a program into classes and designing relationships between them takes more effort than writing simple scripts.
- ❖ **Difficulty in Debugging** Since methods may belong to different objects interacting dynamically, debugging OOP code can be more challenging compared to linear procedural code.
- ❖ **Learning Curve** Beginners may struggle with concepts like encapsulation, abstraction, inheritance, and polymorphism, making OOP harder to grasp initially.

iv. OOP Concepts

In Object-Oriented Programming (OOP), there are four fundamental pillars/concepts that define its structure and principles. These pillars help in building modular, reusable, and scalable applications in VB.NET. Let's explore each one of them respectively;

2.2. Encapsulation – Data Protection & Security

Encapsulation means hiding the internal details of an object and exposing only necessary parts. This helps in data security and ensures controlled access to variables and methods.

Example: Encapsulation

```
Public Class BankAccount  
Private Balance As Decimal ' Private ensures restricted access  
  
Public Sub Deposit(amount As Decimal)  
    If amount > 0 Then  
        Balance += amount  
    End If  
End Sub  
  
Public Function GetBalance() As Decimal  
    Return Balance ' Only accessible via this method  
End Function  
End Class
```

Real-Life Analogy: Think of an ATM machine—it hides internal processing, but allows users to deposit and check their balance through secure interactions.

Code Explanation

- ❖ A class named **BankAccount** is created. Which acts as a blueprint for objects that represent bank accounts.

- ❖ **Balance** is a private variable, meaning it can only be accessed inside the class. It stores the amount of money in the account. *(The current value of Balance is added to the value of amount. The result is then stored back into Balance.)*
- ❖ **Deposit (amount As Decimal)** is a public method that allows users to add money to the account. If the deposit amount is greater than zero, it is added to Balance.
- ❖ This method allows users to retrieve their current balance. Since Balance is private, this method provides a controlled way to check the balance.

Question 7: What Are Getters and Setters?

Getters and Setters are special methods that control access to an object's private data, following the principle of encapsulation. Whereby;

- ❖ **Getters (Get methods)** are used to retrieve the value of a private variable.
- ❖ **Setters (Set methods)** are used to update the value of a private variable, often with validation.

Encapsulation prevents direct access to sensitive data, allowing controlled access through getters and setters.

Public Class BankAccount

Private Balance As Decimal ' Private data: Cannot be accessed directly

' Getter Method

Public Function GetBalance() As Decimal

Return Balance ' Returns the private Balance value

End Function

' Setter Method

Public Sub SetBalance(newBalance As Decimal)

If newBalance >= 0 Then ' Validation: Balance cannot be negative

Balance = newBalance

Else

Console.WriteLine("Invalid balance amount!")

End If

End Sub

End Class

Dim myAccount As New BankAccount()

' Setting the balance through a setter

myAccount.SetBalance(500)

' Retrieving the balance through a getter

Console.WriteLine("Current Balance: " & myAccount.GetBalance())

Class Person

Private name As String

Private age As Integer

```
' Getter for name
Public Function GetName() As String
    Return name
End Function

' Setter for name
Public Sub SetName(value As String)
    If value <> "" Then
        name = value
    Else
        Console.WriteLine("Name cannot be empty!")
    End If
End Sub

' Getter for age
Public Function GetAge() As Integer
    Return age
End Function

' Setter for age
Public Sub SetAge(value As Integer)
    If value > 0 Then
        age = value
    Else
        Console.WriteLine("Age must be a positive number!")
    End If
End Sub
End Class

Module Module1
    Sub Main()
        Dim person1 As New Person()

        person1.SetName("Alice")
        person1.SetAge(25)

        Console.WriteLine("Name: " & person1.GetName())
        Console.WriteLine("Age: " & person1.GetAge())
    End Sub
End Module
```

Explanation

- ❖ The **name** and **age** variables are **private**, meaning they cannot be accessed directly from outside the class.
- ❖ Encapsulation ensures that these values are protected from unintended modifications.
- ❖ This **getter method** allows access to the name variable.
- ❖ Instead of accessing name directly, the program uses **GetName()** to retrieve its value

- ❖ The setter method **SetName(value As String)** allows modification of name but with validation:
- ❖ If the given value is not an empty string, it updates the name.
- ❖ Otherwise, it prints "Name cannot be empty!" to prevent invalid input.
- ❖ This retrieves the age value using **GetAge()** instead of directly accessing the variable.
- ❖ The setter method **SetAge(value As Integer)** ensures that only valid ages are set:
- ❖ If the age is greater than **0**, it updates age.
- ❖ Otherwise, it prints "**Age must be a positive number!**".
- ❖ Creates an instance of the Person class named **person1**.
- ❖ Calls the setter methods to assign "**Alice**" to name and **25** to age.
- ❖ Calls the getter methods to fetch values and display them.

Analogy: Think of the ATM machine:

- ❖ **Getter:** When you check your balance, the ATM displays your account balance.
- ❖ **Setter:** When you deposit money, the ATM validates the amount before updating.

Exercise 15: Create a **User class** where the password can be set, but not directly read back. We will also ensure the username is never empty. Using the **_** prefix (e.g., **_username**) to hide data. Use Getters and Setters to control how data is handled. And check if the data is valid before saving it.

Exercise 16: Create a **Thermostat class** that restricts the temperature to a safe range (**between 15°C and 30°C**) preventing "impossible" values. The class decides if the change is allowed, not the user.

2.3. Inheritance – Code Reusability

Inheritance allows a class (**child**) to inherit attributes and methods from another class (**parent**). This eliminates duplicate code and promotes reusability.

Example: Inheritance

```
Public Class Vehicle
  Public Brand As String
  Public Model As String

  Public Sub Start()
    Console.WriteLine("Vehicle started!")
  End Sub
End Class

Public Class Car
  Inherits Vehicle ' Car inherits from Vehicle

  Public Sub Honk()
    Console.WriteLine("Car honks: Beep Beep!")
  End Sub
End Class

Dim myCar As New Car()
```

```
myCar.Brand = "Toyota"  
myCar.Start() ' Inherited from Vehicle  
myCar.Honk() ' Specific to Car
```

Real-Life Analogy: A child inherits characteristics (**like eye color, height**) from parents, but also develops unique traits.

Exercise 17: Create a general **Employee class** for basic info, and a **Manager class** that **inherits** from it but adds a **Bonus feature**.

Exercise 18: Create a base class called **Animal** with a **Name** property. Then, create two child classes, **Dog and Cat**, that **inherit** that property but have their own **unique sounds**.

Types of Inheritance in OOP

There are several types of inheritance, but VB.NET supports only some of them, the first on the list is **Single Inheritance** which is used in the example above, whereby the **child class** inherits from one **parent class**

Second on our list is **Multi-Level Inheritance** whereby a **derived class** inherits from another **derived class**, forming a chain.

Example: Multi-Level Inheritance

```
Public Class LivingBeing  
    Public Sub Exist()  
        Console.WriteLine("I am alive")  
    End Sub  
End Class  
  
Public Class Mammal  
    Inherits LivingBeing  
  
    Public Sub Breathe()  
        Console.WriteLine("I breathe air")  
    End Sub  
End Class  
  
Public Class Human  
    Inherits Mammal  
  
    Public Sub Speak()  
        Console.WriteLine("Humans can speak")  
    End Sub  
End Class  
  
Dim person As New Human()  
person.Exist() ' Inherited from LivingBeing  
person.Breathe() ' Inherited from Mammal  
person.Speak() ' Own method
```

Real Life Analogy: Grandfather → Father → Son: Each generation inherits traits from the previous one.

And last on the list we have got **Hierarchical Inheritance** whereby, **multiple child classes inherit from a single parent class.**

Example: Hierarchical Inheritance

```
Public Class Vehicle
    Public Sub StartEngine()
        Console.WriteLine("Engine started")
    End Sub
End Class

Public Class Car
    Inherits Vehicle

    Public Sub OpenTrunk()
        Console.WriteLine("Car trunk opened")
    End Sub
End Class

Public Class Motorcycle
    Inherits Vehicle

    Public Sub KickStart()
        Console.WriteLine("Motorcycle kick-started")
    End Sub
End Class

Dim myCar As New Car()
myCar.StartEngine() ' Inherited method
myCar.OpenTrunk() ' Car-specific

Dim myBike As New Motorcycle()
myBike.StartEngine() ' Inherited method
myBike.KickStart() ' Motorcycle-specific
```

Real-Life Analogy: A parent has multiple children; each child inherits the parent's characteristics but has its own unique features.

Multiple Inheritance (Not Supported in VB.NET)

A class inherits from more than one parent class. VB.NET does NOT support multiple inheritance directly but provides an alternative using Interfaces.

Example (Using Interfaces instead of Multiple Inheritance):

```
Public Interface IPrintable
    Sub PrintDocument()
End Interface
```

```
Public Interface IScannable  
Sub ScanDocument()  
End Interface  
  
Public Class PrinterScanner  
Implements IPrintable, IScannable  
  
Public Sub PrintDocument() Implements IPrintable.PrintDocument  
Console.WriteLine("Printing document..")  
End Sub  
  
Public Sub ScanDocument() Implements IScannable.ScanDocument  
Console.WriteLine("Scanning document..")  
End Sub  
End Class  
  
Dim myDevice As New PrinterScanner()  
myDevice.PrintDocument()  
myDevice.ScanDocument()
```

Real-Life Analogy: A child inherits traits from both parents, but VB.NET avoids direct multiple inheritance to prevent conflicts.

Hybrid Inheritance (Not Supported in VB.NET)

Hybrid inheritance is a mix of different types (combination of hierarchical, multiple, and single inheritance). VB.NET does NOT support hybrid inheritance directly but can mimic it using interfaces.

2.4. Polymorphism – Flexibility & Extensibility

Polymorphism means one interface, multiple implementations. It allows **method overriding** and **method overloading**, making code more dynamic.

Method Overloading Example: (same method name, different parameters)

```
Public Class MathOperations  
Public Function Add(x As Integer, y As Integer) As Integer  
Return x + y  
End Function  
Public Function Add(x As Double, y As Double) As Double  
Return x + y ' Same method name, different parameters  
End Function  
End Class
```

Method Overriding Example: (child class modifies behaviour of parent class method)

```
Public Class Animal  
  Public Overridable Sub MakeSound()  
    Console.WriteLine("Animal makes a sound")  
  End Sub  
End Class  
  
Public Class Dog  
  Inherits Animal  
  
  Public Overrides Sub MakeSound()  
    Console.WriteLine("Dog barks: Woof Woof!")  
  End Sub  
End Class
```

Real-Life Analogy: A light switch can turn on different types of lights—LED, fluorescent, or halogen bulbs—same action, different responses.

Exercise 19: Create a base class **Instrument** with a method **Play()**. Then, create **Guitar** and **Piano** classes that change how **Play()** sounds.

2.5. Abstraction – Simplifying Complexity

Abstraction hides unnecessary details and exposes only essential features to the user, making programs more efficient.

Example: Abstraction

```
Public MustInherit Class PaymentProcessor  
  Public MustOverride Sub MakePayment(amount As Decimal)  
End Class  
  
Public Class CreditCardPayment  
  Inherits PaymentProcessor  
  
  Public Overrides Sub MakePayment(amount As Decimal)  
    Console.WriteLine("Processing credit card payment of $" & amount)  
  End Sub  
End Class
```

Real-Life Analogy: A car engine, drivers only use the steering wheel and pedals, while the complex mechanics are hidden under the hood.

Question 8: What are abstract classes and interfaces as used in Abstraction

Abstract classes and interfaces are two mechanisms that support abstraction, helping define the structure of objects without revealing implementation details. However, they differ in how they achieve abstraction and in their usage.

Abstract Class in VB.NET

An abstract class in VB.NET is created using the **MustInherit** keyword. It cannot be instantiated (**That is you cannot create an object directly from that class.**) and serves as a blueprint for other classes. It can have both abstract methods (declared using **MustOverride**) and concrete methods (regular methods with implementation).

Example of an Abstract Class in VB.NET:

```
Public MustInherit Class Animal
    ' Abstract method (must be implemented in derived class)
    Public MustOverride Sub MakeSound()

    ' Concrete method (has implementation)
    Public Sub Sleep()
        Console.WriteLine("Sleeping...")
    End Sub
End Class

Public Class Dog
    Inherits Animal

    ' Implementing the abstract method
    Public Overrides Sub MakeSound()
        Console.WriteLine("Barks")
    End Sub
End Class
```

- ❖ **Animal** is an **abstract class** that defines **MakeSound()** without implementation.
- ❖ **Dog** inherits from **Animal** and provides its own implementation.

Interface in VB.NET

An interface in VB.NET defines a contract that other classes must follow. Unlike abstract classes, interfaces only declare methods without any implementation. They are created using the Interface keyword.

Example of an Interface in VB.NET:

```
Public Interface IAnimal
    Sub MakeSound() ' Abstract method (no implementation)
End Interface

Public Class Dog
    Implements IAnimal

    ' Implementing the interface method
    Public Sub MakeSound() Implements IAnimal.MakeSound
        Console.WriteLine("Barks")
    End Sub
End Class
```

- ❖ **IAnimal** is an interface that declares **MakeSound()**, but does not implement it.
- ❖ **Dog** implements **IAnimal**, providing its own definition.

In VB.NET, we use the keyword **MustInherit** for the class and **MustOverride** for the methods. An abstract class is like a ghost you cannot create an object directly from it (you can't say **New Shape()**), you can only create objects from its children (like **New Circle()**).

Both abstract classes and interfaces allow abstraction by hiding implementation details, ensuring that developers focus on defining behaviours rather than their internal mechanics.

Key Differences Between Abstract Classes and Interfaces in VB.NET

Feature	Abstract Class (MustInherit)	Interface (Interface)
Method Types	Abstract (MustOverride) + Concrete	Only Abstract (until default methods in VB.NET)
Variables	Can have instance variables	No instance variables (only constants)
Inheritance	Single (Inherits)	Multiple (Implements)

Exercise 20: Create an abstract **class Shape**. Every shape has an area, but the formula for a Square is different from a Circle. We "**force**" every child class to create its own **GetArea** method.

Exercise 21: Create an abstract class **Appliance**. Every appliance can be **turned on**, but "**turning on**" a **Toaster** is very different from "**turning on**" a **Refrigerator**.

2.6. EXCEPTION

In VB.NET, an **exception** is an error that occurs during the execution of a program, **disrupting its normal flow**. **Exception handling** allows *developers to anticipate, detect, and manage errors gracefully, preventing the program from crashing unexpectedly*.

Question 9: What therefore causes those exceptions?

Exceptions can occur due to:

- ❖ Invalid Input (e.g., dividing by zero)
- ❖ File Not Found (e.g., accessing a missing file)
- ❖ Null References (e.g., trying to use an uninitialized object)
- ❖ Index Out of Bounds (e.g., accessing an array element beyond its limit)

Handling Exceptions in VB.NET

VB.NET provides structured exception handling using **Try...Catch** blocks.

Example: Handling Division by Zero

```
Module Program
    Sub Main()
        Try
            Dim num As Integer = 10
            Dim divisor As Integer = 0
```

```
Dim result As Integer = num / divisor ' This will cause an exception  
Catch ex As DivideByZeroException  
Console.WriteLine("Error: Cannot divide by zero!")  
Finally  
Console.WriteLine("Execution completed.")  
End Try  
End Sub  
End Module
```

How It Works:

- ❖ **Try Block:** Contains code that may cause an exception.
- ❖ **Catch Block:** Catches specific exceptions and handles errors.
- ❖ **Finally Block:** Runs cleanup code (e.g., closing files), whether an exception occurs or not.

Common Exception Types in VB.NET

Exception Type	Description
DivideByZeroException	Occurs when dividing by zero
NullReferenceException	Trying to use an uninitialized object
IndexOutOfRangeException	Accessing an invalid array index
FileNotFoundException	Trying to access a missing file
InvalidOperationException	Performing an operation that is not allowed

Exercise 22: Demonstrate how the **NullReferenceException** is used

Exercise 23: Demonstrate how the **IndexOutOfRangeException** is used

Exercise 24: Demonstrate how the **FileNotFoundException** is used

Exercise 25: Demonstrate how the **InvalidOperationException** is used

MODULE 03

INTRODUCTION TO WINDOWS FORMS APPLICATION

3. INTRODUCTION

A Windows Forms Application or (**WinForms for short**) in VB.NET is used to create Graphical User Interface (GUI) applications that run on Windows. It provides a visual framework for designing forms, adding controls, and handling events. A Windows Form is like a blank window that you can add controls to:

- ❖ Buttons
- ❖ Labels
- ❖ TextBoxes
- ❖ CheckBoxes
- ❖ Menus, and more.

You design these using drag-and-drop tools in Visual Studio, and write code to make things happen when users interact with them.

Key Features of Windows Forms Applications

- ❖ Drag-and-Drop Design – Easily build the interface using a visual designer.
- ❖ Built-in Controls – Buttons, textboxes, labels, and other UI elements.
- ❖ Event-Driven Programming – Code runs based on user interactions like clicks.
- ❖ Custom Styling – Modify colours, fonts, and layouts.
- ❖ Database Integration – Connect to databases for data management.

Example in VB.NET:

Let's say you want a form with a button that says "**Click Me**", and when clicked, it shows a message.

Steps:

- ❖ Open Visual Studio → New Project → Windows Forms App (VB.NET)
- ❖ Drag a Button from the toolbox to the form
- ❖ Double-click the button to write code for its Click event

```
Public Class Form1  
    Private Sub Button1_Click(sender As Object, e As EventArgs) Handles Button1.Click  
        MessageBox.Show("Hello! You clicked the button.")  
    End Sub  
End Class
```

3.1. Create a picture viewer Windows Forms App (.NET Framework)

Create your Windows Forms project

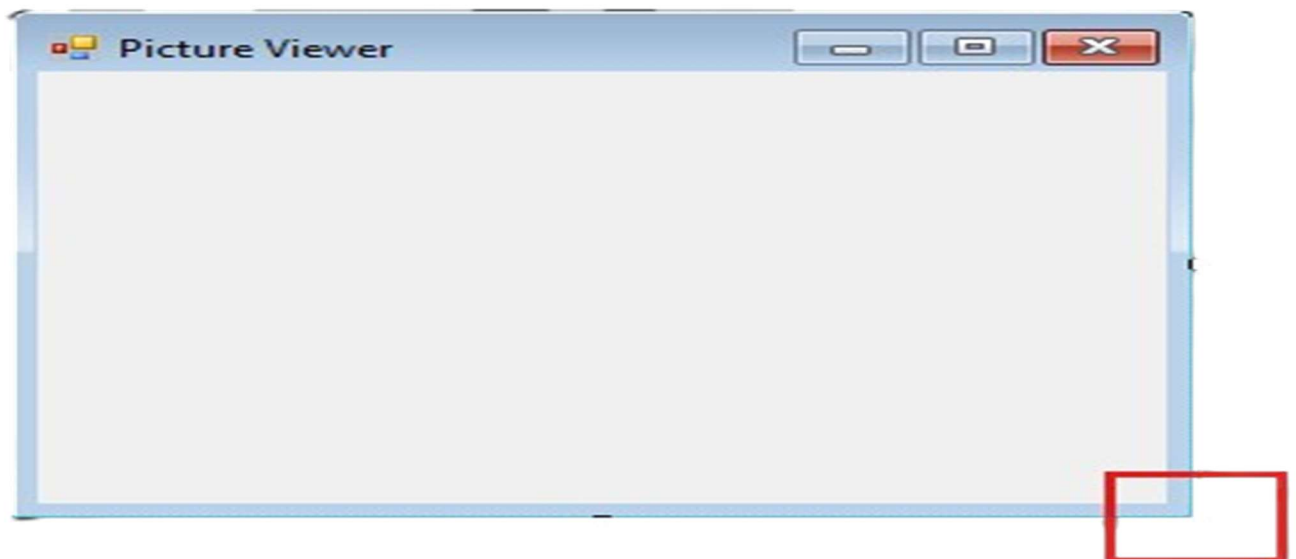
When you create a picture viewer, the first step is to create a Windows Forms App project.

- ❖ Open Visual Studio.
- ❖ On the start window, select Create a new project.

Add a layout element

Your picture viewing app contains *a picture box, a checkbox, and four buttons*, which shall be added later. The layout element controls their location on the form. This section shows you how to change the title of your form, resize the form, and add a layout element.

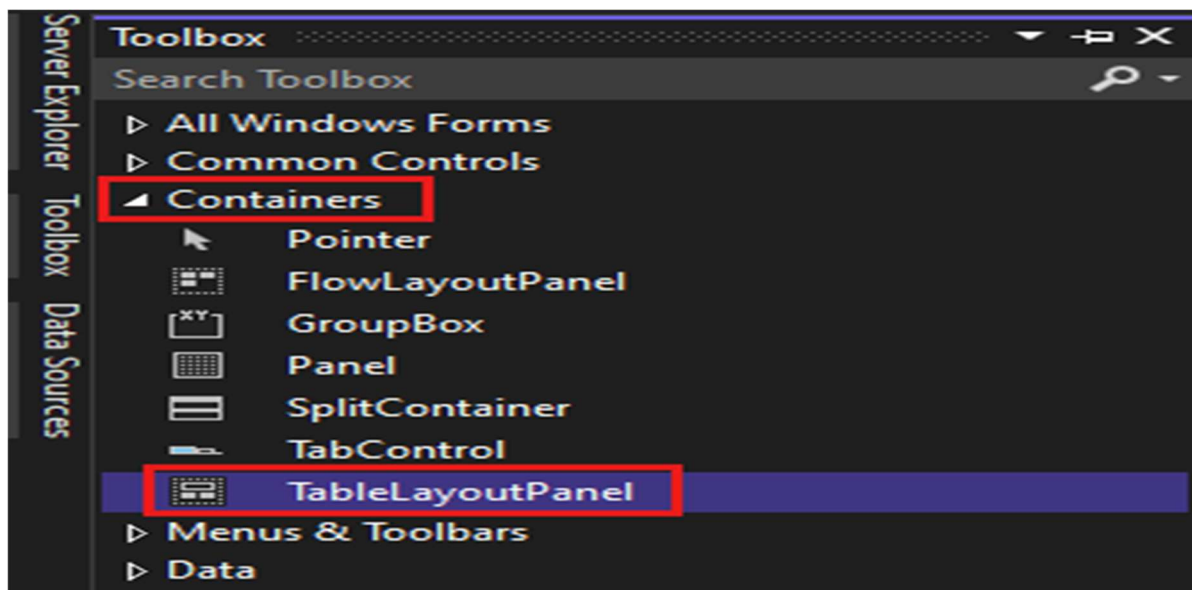
- ❖ In your project, select the Windows Forms Designer. The tab reads Form1.vb [Design] for Visual Basic.
- ❖ Select anywhere in Form1.
- ❖ The Properties window now displays properties for the form. The Properties window is usually in the lower right of Visual Studio. This section controls various properties, such as foreground and background colour, title text that appears at the top of the form, and the size of the form.
- ❖ If you don't see Properties, select View > Properties Window.
- ❖ Find the Text property in the Properties window. Depending on how the list is sorted, you might need to scroll down. Enter the value Picture Viewer, and then choose Enter.
- ❖ Your form now has the text Picture Viewer in its title bar.
- ❖ Select the form, again. Select the form's lower-right drag handle. The handle is a small white square in the lower-right corner of the form.



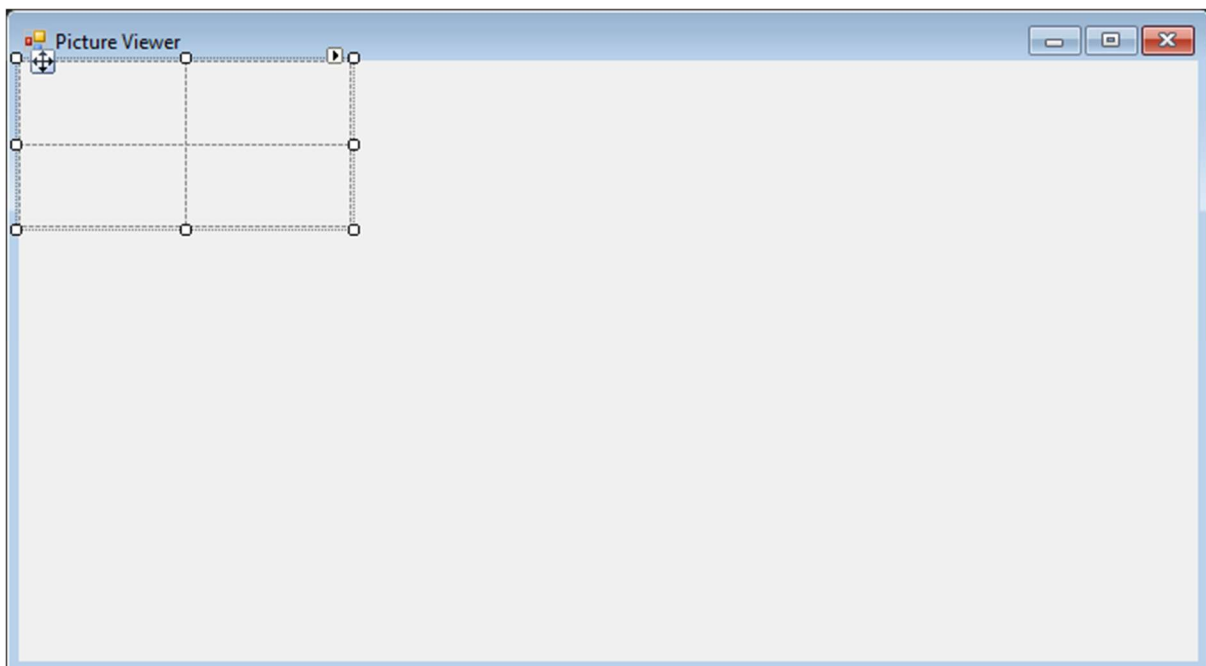
Drag the handle to resize the form so the form is wider and a bit taller. If you look at the Properties window, the Size property is different. You can also change the size of the form by changing the Size property.

- ❖ On the left side of the Visual Studio IDE, select the Toolbox tab. If you don't see it, select View > Toolbox from the menu bar or Ctrl+Alt+X.

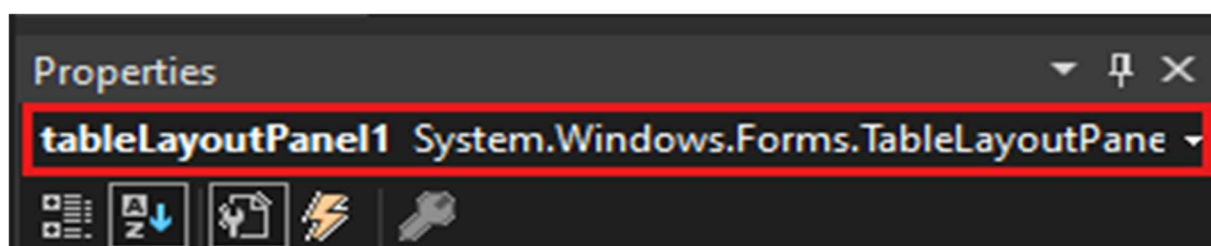
- ❖ Select the small triangle symbol next to Containers to open the group.



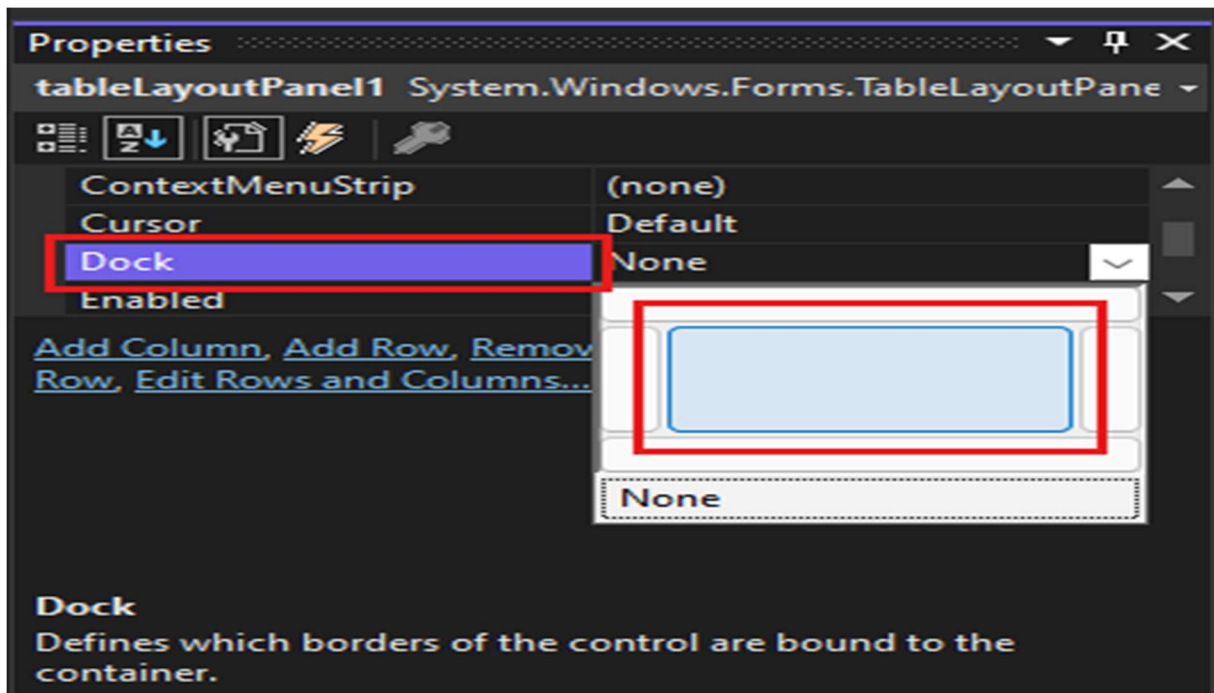
- ❖ Double-click **TableLayoutPanel** in the Toolbox. You can also drag a control from the toolbox onto the form. The **TableLayoutPanel** control appears in your form.



- ❖ Select the TableLayoutPanel. You can verify what control is selected by looking at the Properties window.



- ❖ With the **TableLayoutPanel** selected, find the **Dock** property, under **property** which has the value **None**. Select the dropdown arrow and then select **Fill**, which is the large button in the middle of the dropdown menu.

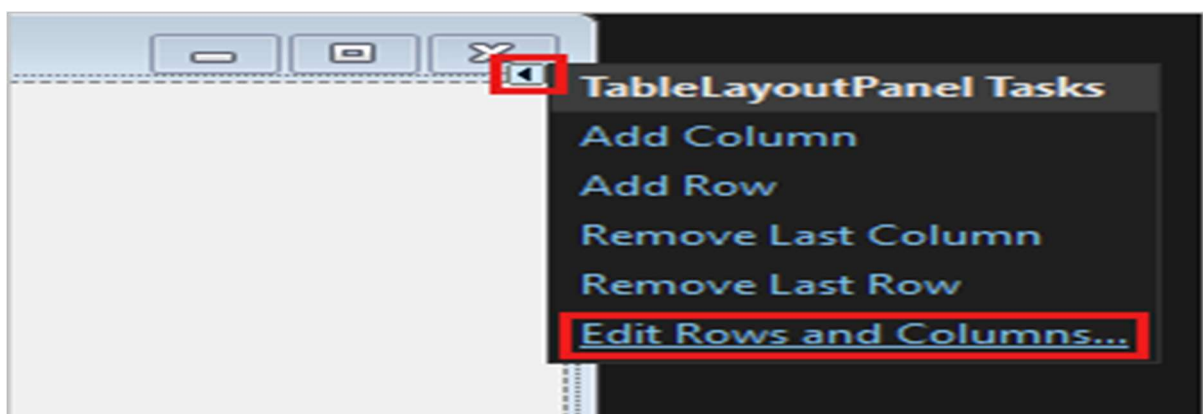


Docking refers to how a window is attached to another window or area.

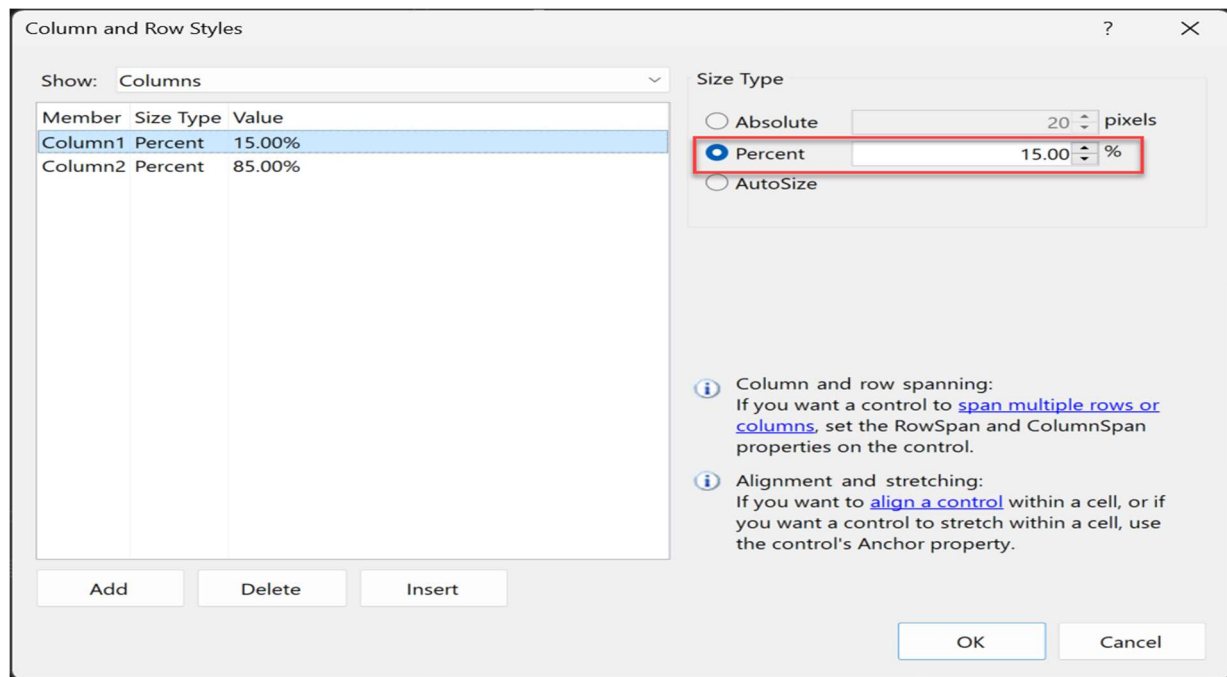
The **TableLayoutPanel** now fills the entire form. If you resize the form again, the **TableLayoutPanel** stays docked, and resizes itself to fit.

- ❖ In the form, select the **TableLayoutPanel**. In the upper-right corner, there's a small black triangle button.

Select the triangle to display the control's task list.

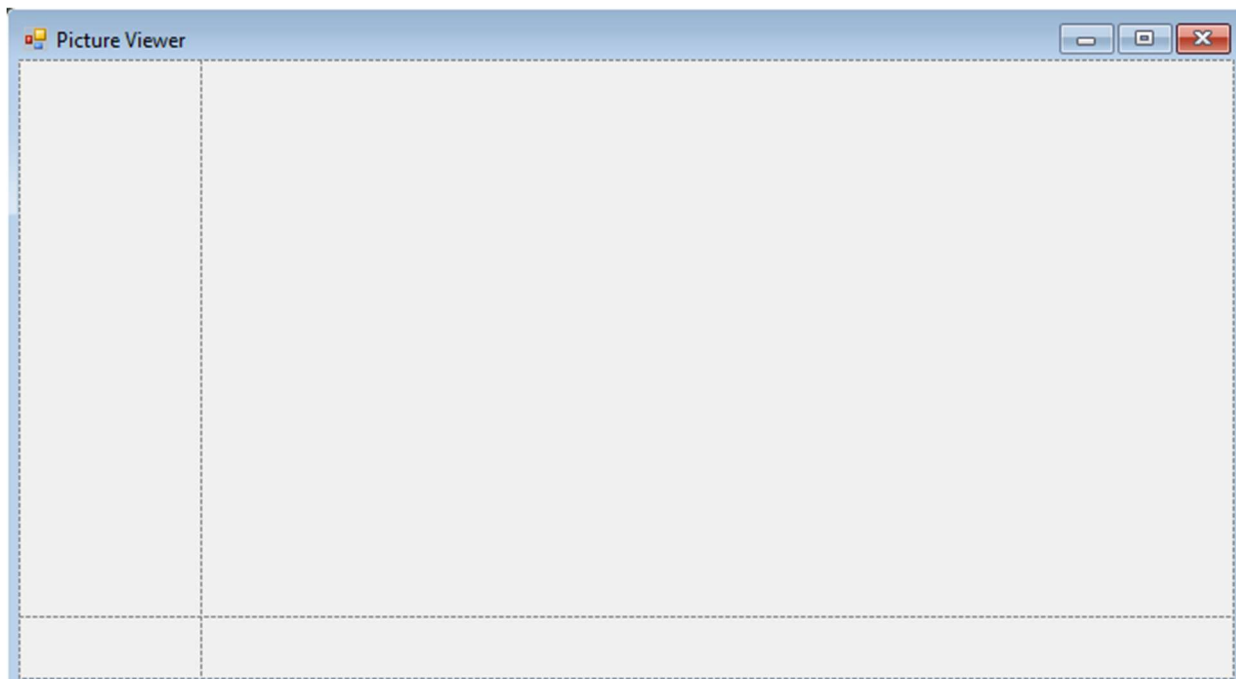


- ❖ Select **Edit Rows and Columns** to display the **Column and Row Styles** dialog box.
- ❖ Select **Column1** and set its size to **15 percent**. Be sure the **Percent** option is selected.
- ❖ Select **Column2** and set it to **85 percent**.



- ❖ From Show at the top of the Column and Row Styles dialog box, select Rows. Set Row1 to 90 percent and Row2 to 10 percent. Select OK to save your changes.

Your **TableLayoutPanel** now has a large top row, a small bottom row, a small left column, and a large right column.

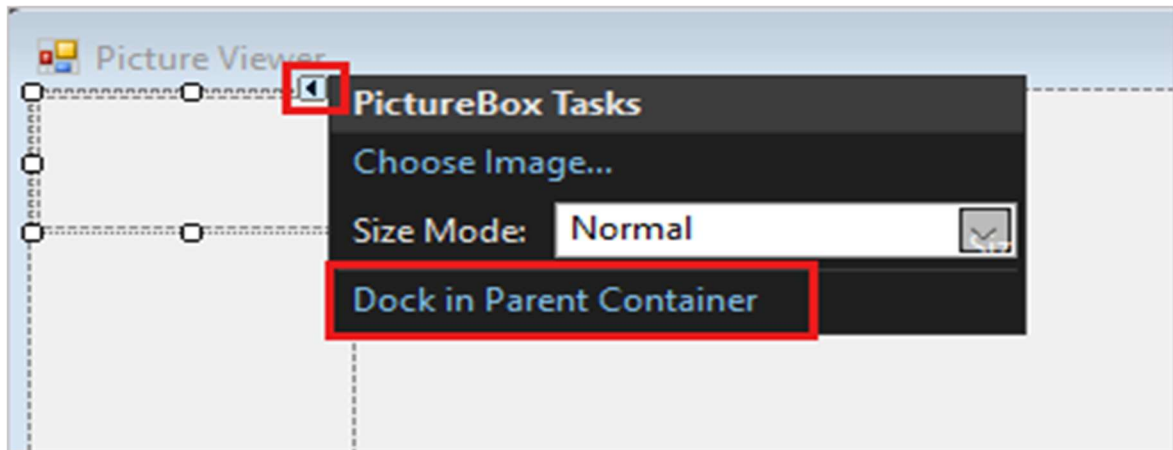


Your layout is complete.

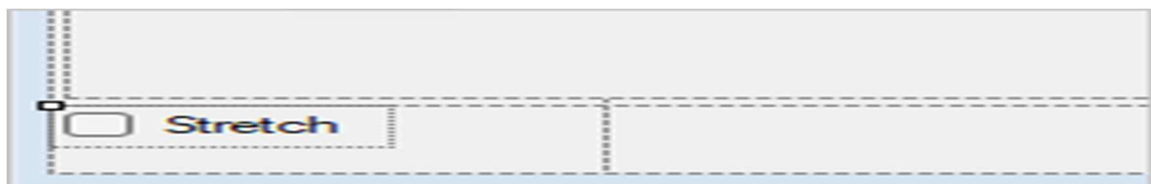
3.1.1. Add controls to the picture viewer Windows Forms App (.NET Framework)

Add controls to your application

- ❖ The Picture Viewer app uses a **PictureBox** control to display a picture. It uses a **CheckBox** and several buttons to manage the picture and background and to close the app. You add the **PictureBox** and **CheckBox** from the Toolbox in the Visual Studio IDE.
- ❖ Open Visual Studio. Your Picture Viewer project appears under Open recent.
- ❖ In the Windows Forms Designer, select the **TableLayoutPanel** you added previously. Check that `tableLayoutPanel1` appears in the Properties window.
- ❖ On the left side of the Visual Studio IDE, select the Toolbox tab. If you don't see it, select **View > Toolbox** from the menu bar or **Ctrl+Alt+X**. In the toolbox, expand **Common Controls**.
- ❖ Double-click **PictureBox** to add a **PictureBox** control to your form. Visual Studio adds the **PictureBox** control to the first empty cell of the **TableLayoutPanel**.
- ❖ Choose the new **PictureBox** control to select it, and then select the black triangle on the new **PictureBox** control to display its task list.



- ❖ Select **Dock in Parent Container**, which sets the **PictureBox Dock** property to **Fill**. You can see that value in the Properties window.
- ❖ In the Properties window for the PictureBox, set the **ColumnSpan** property to **2**. The PictureBox now fills both columns.
- ❖ **Set the BorderStyle property to Fixed3D.**
- ❖ In the Windows Forms Designer, select **TableLayoutPanel**. Then, in the Toolbox, double-click the **CheckBox** item to add a new **CheckBox** control. Your PictureBox takes up the first two cells in the TableLayoutPanel, so the **CheckBox** control is added to the **lower-left** cell.
- ❖ Choose the **Text** property and enter **Stretch**.

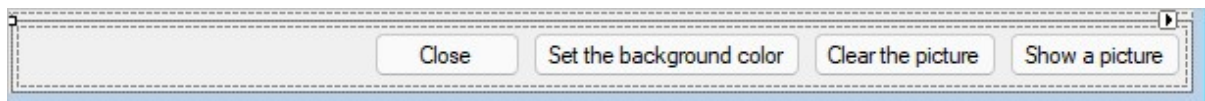


Add buttons in a layout panel

- ❖ So far, you added controls to the **TableLayoutPanel**. These steps show you how to add **four buttons** to a new layout panel in the **TableLayoutPanel**.
- ❖ Select the **TableLayoutPanel** on the form. Open the **Toolbox**, and select **Containers**. Double-click **FlowLayoutPanel** to add a new control to the last cell of the **TableLayoutPanel**.
- ❖ Set the **FlowLayoutPanel's Dock property to Fill**. You can set this property by selecting the black triangle, then selecting Dock in parent container.

A **FlowLayoutPanel** is a container that arranges other controls in a row, one after another.

- ❖ Select the new **FlowLayoutPanel**, and then open the **Toolbox** and select **Common Controls**. Double-click the **Button item** to add a button control called **button1**.
- ❖ Double-click Button again to add another button. The IDE calls the next one button2.
- ❖ Add two more buttons this way. Another option is to select button2. Then select Edit > Copy or Ctrl+C. Next, choose Edit > Paste from the menu bar or press Ctrl+V to paste a copy of your button. Now paste it again. The IDE adds button3 and button4 to the FlowLayoutPanel.
- ❖ Select the first button and set its Text property to Show a picture.
- ❖ Set the Text properties of the next three buttons to **Clear the picture, Set the background color, and Close**.
- ❖ To size the buttons and arrange them, select the **FlowLayoutPanel**. Set the **FlowDirection** property to **RightToLeft**.
- ❖ The buttons should align themselves to the right side of the cell, and reverse their order so that the Show a picture button is on the right of the cell. You can drag the buttons around the **FlowLayoutPanel** to arrange them in any order.
- ❖ Choose the Close button to select it. Then, to select the rest of the buttons at the same time, press and hold the Ctrl key and choose them, too.
- ❖ In the Properties window, set the **AutoSize** property to **True**. The buttons resize to fit their text.

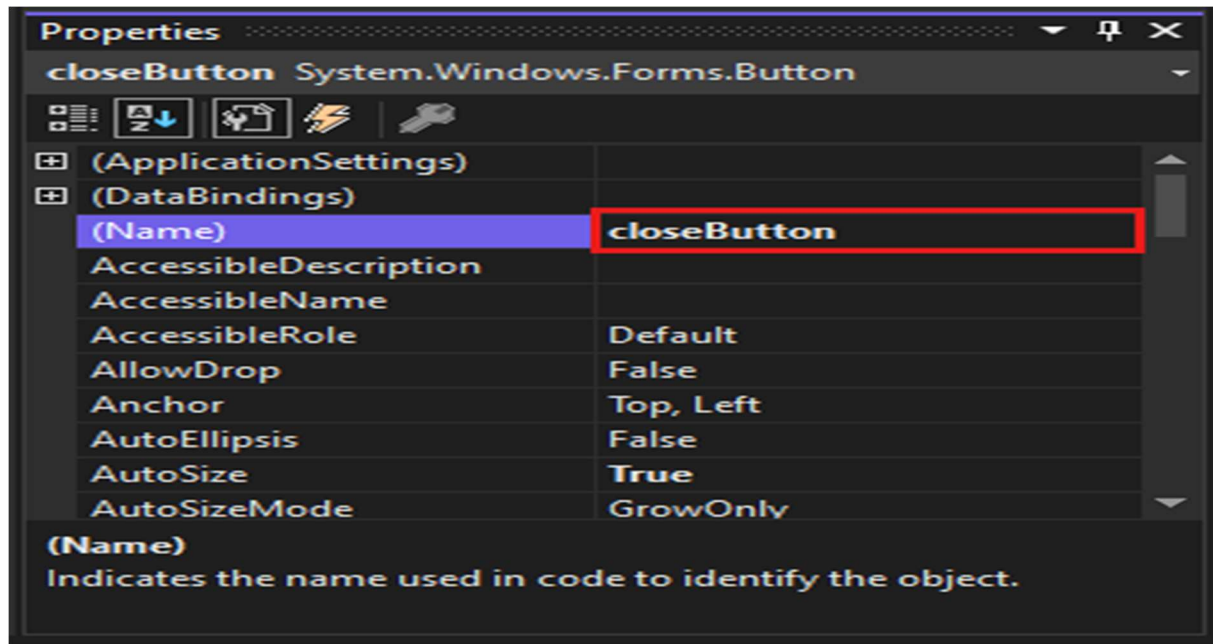


You can run your program to see how the controls look. Select the F5 key, select Debug > Start Debugging, or select the Start button. The buttons that you added don't do anything, yet.

Change control names

There are four buttons on your form, named **Button1, Button2, Button3, and Button4 in VB**. In Visual Basic, the default first letter of any control name is capitalized, so the buttons are named Button1, Button2, Button3, and Button4. Use these steps to give them more informative names.

- ❖ On the form, choose the Close button. If you still have all the buttons selected, choose Esc to cancel the selection.
- ❖ In the Properties window, look for (Name). Change the name to **closeButton**.



The IDE doesn't accept names that contain spaces.

- ❖ Rename the other three buttons to *backgroundButton*, *clearButton*, and *showButton*. You can verify the names by choosing the control selector dropdown list in the Properties window. The new button names appear.

You can change the name of any control, such as the `TableLayoutPanel` or `CheckBox`.

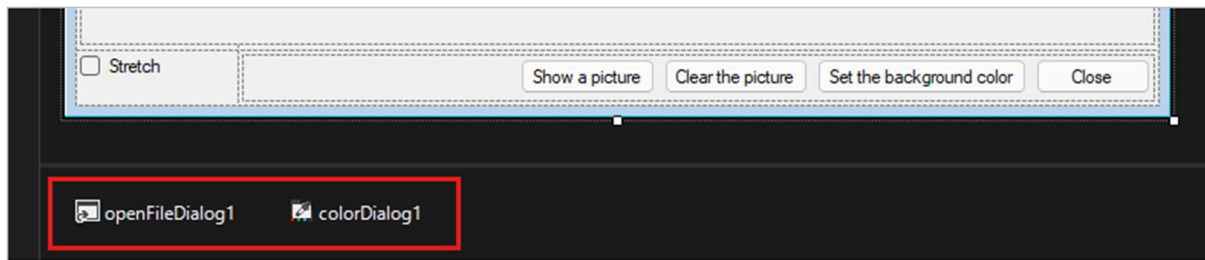
Add dialog components

Your app can open picture files and choose a background color by using components. **A component is like a control.** You use the Toolbox to add a component to your form. You set its properties using the Properties window.

Unlike a control, adding a component to your form doesn't add a visible item. Instead, it provides certain behaviors that you can trigger with code. For instance, it's a component that opens an Open File dialog box.

In this section, you add an **OpenFileDialog** component and a **ColorDialog** component to your form.

- ❖ Select the Windows Forms Designer Then open the Toolbox and select the **Dialogs** group.
- ❖ Double-click **OpenFileDialog** to add a component called **openFileDialog1** to your form.
- ❖ Double-click **ColorDialog** to add a component called **colorDialog1**. The components appear at the bottom of Windows Forms Designer as **icons**.



Choose the openFileDialog1 icon and set two properties:

Set the Filter property to the following value:

JPEG Files (*.jpg)|*.jpg|PNG Files (*.png)|*.png|BMP Files (*.bmp)|*.bmp|All files (*.*)|*.*

- ❖ Set the Title property to the following value: **Select a picture file.**

The Filter property settings specify the types that the Select a picture dialog box displays.

3.1.2. Add code to the picture viewer Windows Forms App

Add event handlers for your controls

In this section, add **event handlers** for the controls that you added in the previous exercise, your application calls an event handler when an action takes place, such as selecting a button, methods, including event handlers, can have any name that you want. When you add an event handler with the IDE, it creates a name based on the control's name and the event being handled.

For example, the Click event for a button named **showButton** is called **showButton_Click()** or **ShowButton_Click()**.

Write code to open a dialog box

The Show a picture button uses the **OpenFileDialog** component to display a picture file. This procedure adds the code used to call that component. The Visual Studio IDE offers a powerful tool called **IntelliSense**. As you type, **IntelliSense** suggests possible code.

- ❖ In Windows Forms Designer, double-click the Show a picture button. The IDE moves your cursor inside the **showButton_Click()** or **ShowButton_Click()** method.
- ❖ Type an *i* on the empty line between **Private Sub...** and **End Sub**. An IntelliSense window opens.
- ❖ The IntelliSense window should highlight the word **if**. Select the Tab key twice to insert the **if** snippet.
- ❖ Select **true** and then type **Op** for Visual Basic.
- ❖ Select Tab to add **openFileDialog1**.
- ❖ Type a period (.) or dot, right after **openFileDialog1**. IntelliSense provides all of the **OpenFileDialog** component's properties and methods. Start to type **ShowDialog** and select Tab. The **ShowDialog()** method shows the Open File dialog box.
- ❖ Add parentheses () immediately after the **"g"** in **ShowDialog**. Your code should be **openFileDialog1.ShowDialog()**.
- ❖ For Visual Basic, add a space, and then use a single equal sign (=).

- ❖ Add another space. Use IntelliSense to enter **DialogResult**.
- ❖ Type a **dot** to open the **DialogResult** value in the IntelliSense window. Enter the letter **O** and choose the Tab key to insert **OK**.

You can copy and paste or use IntelliSense to add it. Your final **showButton_Click()** method should look similar to the following code.

```
Private Sub showButton_Click() Handles showButton.Click  
' Show the Open File dialog. If the user clicks OK, load the  
' picture that the user chose.  
If OpenFileDialog1.ShowDialog() = DialogResult.OK Then  
    PictureBox1.Load(OpenFileDialog1.FileName)  
End If  
  
End Sub
```

Note: It's the best practice to always comment your code. Code comments make it easier to understand and maintain your code in the future.

Write code for the other controls

If you run your application now, you can select **Show a picture**. Picture Viewer opens the **Open File dialog box**, where you can select a picture to display.

In this section, add the code for the other event handlers.

- i. In **Windows Forms Designer**, double-click the **Clear the picture button**. For Visual Basic, add the code between **Private Sub** and **End Sub**.

```
Private Sub clearButton_Click() Handles clearButton.Click  
' Clear the picture.  
PictureBox1.Image = Nothing  
End Sub
```

- ii. Double-click the **Set the background color** button and add the code.

```
Private Sub backgroundButton_Click() Handles backgroundButton.Click  
' Show the color dialog box. If the user clicks OK, change the  
' PictureBox control's background to the color the user chose.  
If ColorDialog1.ShowDialog() = DialogResult.OK Then  
    PictureBox1.BackColor = ColorDialog1.Color  
End If  
End Sub
```

- iii. Double-click the **Close button** and add the code.

```
Private Sub closeButton_Click() Handles closeButton.Click  
' Close the form.  
Close()  
End Sub
```

- iv. Double-click the **Stretch checkbox** and add the code.

```
Private Sub CheckBox1_CheckedChanged() Handles CheckBox1.CheckedChanged  
' If the user selects the Stretch check box, change  
' the PictureBox's SizeMode property to "Stretch". If the user  
' clears the check box, change it to "Normal".  
If CheckBox1.Checked Then  
PictureBox1.SizeMode = PictureBoxSizeMode.StretchImage  
Else  
PictureBox1.SizeMode = PictureBoxSizeMode.Normal  
End If  
End Sub
```

Run your application

3.2. Create a timed math quiz (.NET Framework)

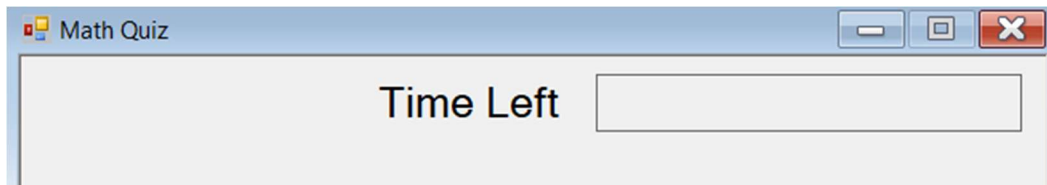
The quiz contains four random math problems that a quiz taker tries to answer within a specified time.

Create the time remaining box

The math quiz contains a box in the upper-right corner. That box shows the number of seconds that remain in the quiz. This section shows you how to use a **label** for that box. You add code for the countdown timer later.

*Note: On the left side of the Visual Studio IDE, select the **Toolbox** tab. If you don't see the toolbox, select **View > Toolbox** from the menu bar or **Ctrl+Alt+X**.*

- i. Select the **Label** control in the **Toolbox**, and then drag it onto the form.
- ii. In the **Properties** box, set the following properties for the label:
 - ❖ Set the (Name) to **timeLabel**.
 - ❖ Change **AutoSize** to **False** so that you can resize the box.
 - ❖ Change the **BorderStyle** to **FixedSingle** to draw a line around the box.
 - ❖ Set the **Size** to **200, 30**.
 - ❖ Select the **Text** property, and then select the **Backspace** key to clear the Text value.
 - ❖ Select the plus sign (+) next to the **Font** property, and then set the **Size** to 15.75.
 - ❖ Move the label to the upper-right corner of the form. When blue spacer lines appear, use them to position the control on the form.
 - ❖ Add another **Label** control from the **Toolbox**, and then set its font size to 15.75.
 - ❖ Set this label's **Text** property to Time Left.
 - ❖ Move the label so that it lines up to the left of the **timeLabel** label.



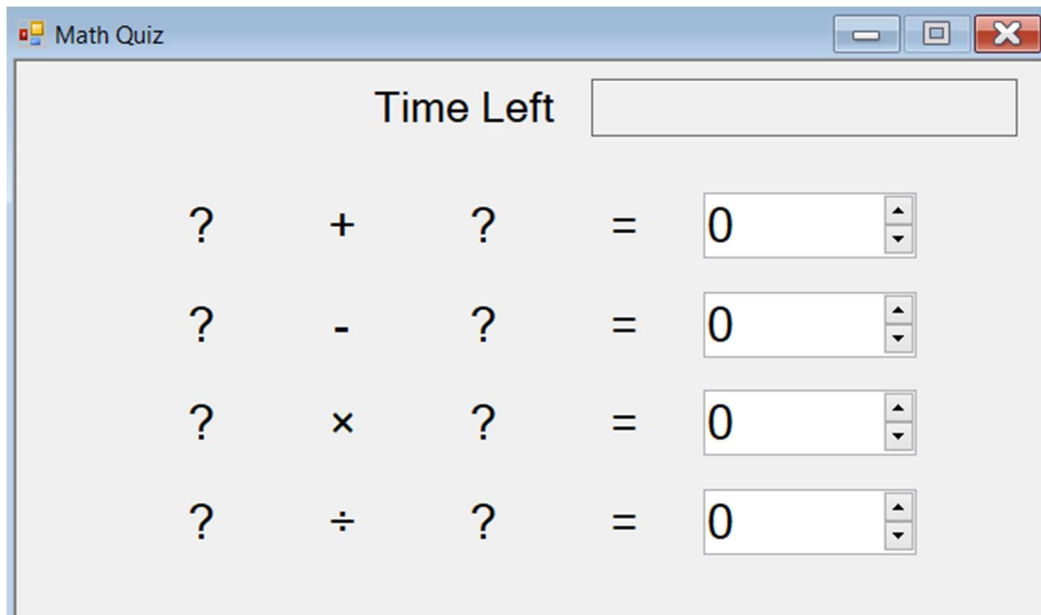
Add controls for the addition problem

The first part of the quiz is an addition problem. This section shows you how to use labels to display that problem.

- i. Add a **Label** control from the **Toolbox** to the form.
- ii. In the **Properties box**, set the following properties for the label:
 - ❖ Set the **Text** to ? (question mark).
 - ❖ Set **AutoSize** to False.
 - ❖ Set the **Size** to 60, 50.
 - ❖ Set the font size to 18.
 - ❖ Set **TextAlign** to **MiddleCenter**.
 - ❖ Set the **Location** to 50, 75 to position the control on the form.
 - ❖ Set the (Name) to **plusLeftLabel**.
- iii. In the form, select the **plusLeftLabel** label that you created. Copy the label by selecting either **Edit > Copy** or **Ctrl+C**.
- iv. Paste the label into the form three times by selecting either **Edit > Paste** or **Ctrl+V** three times.
- v. Arrange the three new labels so that they are in a row to the right of the **plusLeftLabel** label.
- vi. Set the second label's **Text property** to + (plus sign).
- vii. Set the third label's (Name) property to **plusRightLabel**.
- viii. Set the fourth label's **Text** property to = (equals sign).
- ix. Add a **NumericUpDown** control from the **Toolbox** to the form. You learn more about this kind of control later.
- x. In the **Properties box**, set the following properties for the **NumericUpDown** control:
 - ❖ Set the font size to 18.
 - ❖ Under **MaximumSize**, set the **width** to 100.
 - ❖ Set the (Name) to **sum**.
 - ❖ Line up the **NumericUpDown** control with the Label controls for the addition problem.

Add controls for the subtraction, multiplication, and division problems

- i. Next, add labels to the form for the remaining math problems.
- ii. Copy the four Label controls and the **NumericUpDown** control that you created for the addition problem. Paste them into the form.
- iii. Move the new controls to line up below the addition controls.
- iv. In the Properties box, set the following properties for the new controls:
 - ❖ Set the (Name) of the first question-mark label to **minusLeftLabel**.
 - ❖ Set the Text of the second label to - (**minus sign**).
 - ❖ Set the (Name) of the second question-mark label to **minusRightLabel**.
 - ❖ Set the (Name) of the **NumericUpDown** control to **difference**.
- iii. Copy the addition controls, and paste them two more times into the form.
- iv. For the **third row**:
 - ❖ Set the (Name) of the first question-mark label to **timesLeftLabel**.
 - ❖ Set the Text of the second label to × (**multiplication sign**). You can copy the multiplication sign from this tutorial and paste it onto the form.
 - ❖ Set the (Name) of the second question-mark label to **timesRightLabel**.
 - ❖ Set the (Name) of the **NumericUpDown** control to **product**.
- v. For the **fourth row**:
 - ❖ Set the (Name) of the first question-mark label to **dividedLeftLabel**.
 - ❖ Set the Text of the second label to ÷ (**division sign**). You can copy the division sign from this tutorial and paste it onto the form.
 - ❖ Set the (Name) of the second question-mark label to **dividedRightLabel**.
 - ❖ Set the (Name) of the **NumericUpDown** control to **quotient**.



Add a start button and set the tab-index order

This section shows you how to add a start button. You also specify the tabbing order of the controls. This ordering determines how the quiz taker moves from one control to the next by using the Tab key.

- i. Add a **Button** control from the **Toolbox** to the form.
- ii. In the **Properties box**, set the following properties of the button:
 - ❖ Set the (Name) to **startButton**.
 - ❖ Set the Text to **Start the quiz**.
 - ❖ Set the font size to **14**.
 - ❖ Set **AutoSize** to True, which causes the button to automatically resize to fit the text.
 - ❖ Set the **TabIndex** to 0. This value makes the start button the first control to receive the focus.
- iii. **Center** the button near the bottom of the form.

Math Quiz

Time Left

? + ? = 0

? - ? = 0

? × ? = 0

? ÷ ? = 0

Start the quiz

- iv. In the Properties box, set the **TabIndex** property of each **NumericUpDown** control:
- ❖ Set the **TabIndex** of the **sum** control to **1**.
 - ❖ Set the **TabIndex** of the **difference** control to **2**.
 - ❖ Set the **TabIndex** of the **product** control to **3**.
 - ❖ Set the **TabIndex** of the quotient control to **4**.

Run your app

Create a random addition problem

- ❖ In your Visual Studio project, select Windows Forms Designer.
- ❖ Select the form, Form1.
- ❖ On the menu bar, select View > Code. Form1.vb appears, so that you can view the code behind the form.
- ❖ Create a Random object by adding a new statement near the top of the code in Form1.vb.

```
Public Class Form1
  ' Create a Random object called randomizer
  ' to generate random numbers.
  Private randomizer As New Random
```

You can use new statements like this one to create **buttons, labels, panels, OpenFileDialogs, ColorDialogs, SoundPlayers, Randoms, and even forms**. These items are called **objects**.

When you run your program, the form is started. The code behind it creates a Random object and names it **randomizer**.

Your quiz needs variables to store the random numbers that it creates for each problem. Before using variables, you declare them, which means listing their names and data types.

- ❖ Add two integer variables to the form, and name them **addend1** and **addend2** in **Form1.vb**.

You use similar syntax to add an integer variable as you did to add the Random object, as the following code shows.

```
' Create a Random object called randomizer  
' to generate random numbers.  
Private randomizer As New Random  
' These integer variables store the numbers  
' for the addition problem.  
Private addend1 As Integer  
Private addend2 As Integer
```

- ❖ Add a method that's named **StartTheQuiz()**. This method uses the Random object's **Next()** method to generate random numbers for the labels. **StartTheQuiz()** will eventually fill in all the problems and then start the timer, so add this information to the summary comment. The function should look like the following code.

```
''' <summary>  
''' Start the quiz by filling in all of the problems  
''' and starting the timer.  
''' </summary>  
''' <remarks></remarks>  
Public Sub StartTheQuiz()  
    ' Fill in the addition problem.  
    ' Generate two random numbers to add.  
    ' Store the values in the variables 'addend1' and 'addend2'.  
    addend1 = randomizer.Next(51)  
    addend2 = randomizer.Next(51)  
  
    ' Convert the two randomly generated numbers  
    ' into strings so that they can be displayed  
    ' in the label controls.  
    plusLeftLabel.Text = addend1.ToString()  
    plusRightLabel.Text = addend2.ToString()  
  
    ' 'sum' is the name of the NumericUpDown control.  
    ' This step makes sure its value is zero before  
    ' adding any values to it.  
    sum.Value = 0  
End Sub
```

When you use the **Next()** method with a **Random** object, such as when you call **randomizer.Next(51)**, you get a random number that's less than **51**, or between **0** and **50**. This code calls **randomizer.Next(51)** so that the two random numbers add up to an answer that's between **0** and **100**.

Take a closer look at these statements.

```
' Convert the two randomly generated numbers  
' into strings so that they can be displayed  
' in the label controls.  
plusLeftLabel.Text = addend1.ToString()  
plusRightLabel.Text = addend2.ToString()
```

These statements set the **Text** properties of **plusLeftLabel** and **plusRightLabel** so that they display the two random numbers. Label controls display values in text format, and in programming, strings hold text. Each integer's **ToString()** method converts the integer into text that a label can display.

Create random subtraction, multiplication, and division problems

The next step is to declare variables and provide random values for the other math problems.

- ❖ Add **integer variables** for the remaining math problems to your form, after the addition problem variables. The code in **Form1.vb** should look like the following sample.

```
Public Class Form1  
    ' Create a Random object called randomizer  
    ' to generate random numbers.  
    Private randomizer As New Random  
  
    ' These integer variables store the numbers  
    ' for the addition problem.  
    Private addend1 As Integer  
    Private addend2 As Integer  
  
    ' These integer variables store the numbers  
    ' for the subtraction problem.  
    Private minuend As Integer  
    Private subtrahend As Integer  
  
    ' These integer variables store the numbers  
    ' for the multiplication problem.  
    Private multiplicand As Integer  
    Private multiplier As Integer  
  
    ' These integer variables store the numbers  
    ' for the division problem.  
    Private dividend As Integer  
    Private divisor As Integer
```

- ❖ Modify the **StartTheQuiz()** method in Form1.vb by adding the following code, starting with the *"Fill in the subtraction problem"* comment.

```
' Fill in the subtraction problem.  
minuend = randomizer.Next(1, 101)  
subtrahend = randomizer.Next(1, minuend)  
minusLeftLabel.Text = minuend.ToString()  
minusRightLabel.Text = subtrahend.ToString()  
difference.Value = 0  
  
' Fill in the multiplication problem.  
multiplicand = randomizer.Next(2, 11)  
multiplier = randomizer.Next(2, 11)  
timesLeftLabel.Text = multiplicand.ToString()  
timesRightLabel.Text = multiplier.ToString()  
product.Value = 0  
  
' Fill in the division problem.  
divisor = randomizer.Next(2, 11)  
Dim temporaryQuotient As Integer = randomizer.Next(2, 11)  
dividend = divisor * temporaryQuotient  
dividedLeftLabel.Text = dividend.ToString()  
dividedRightLabel.Text = divisor.ToString()  
quotient.Value = 0
```

This code uses the **Next()** method of the Random class a little differently from how the addition problem does. When you give the **Next()** method two values, it picks a random number that's greater than or equal to the first value and less than the second one.

By using the **Next()** method with two arguments, you can ensure the subtraction problem has a positive answer, the multiplication answer is at most **100**, and the division answer isn't a fraction.

Add an event handler to the start button

In here, you will add code to start the quiz when the start button is **clicked**. Code that runs in reaction to an event like a button selection is called an **event handler**.

- ❖ In Windows Forms Designer, either double-click the Start the quiz button, or select it and then select Enter. The form's code appears, and a new method is visible.

These actions add a **Click Event Handler** to the start button. When a quiz taker selects this button, the app runs the code that you'll add to this new method.

- ❖ Add the following two statements so that the event handler starts the quiz.

```
' Call the StartTheQuiz() method and enable  
' the Start button.  
Private Sub startButton_Click() Handles startButton.Click  
    StartTheQuiz()  
    startButton.Enabled = False  
End Sub
```

The first statement calls the new **StartTheQuiz()** method. The second statement sets the **Enabled property** of the **startButton** control to **false** so that the quiz taker can't select the button during a quiz.

Run your app

- ❖ Save your code.
- ❖ Run your app, and then select Start the quiz. Random math problems appear, as the following screenshot shows.

Math Quiz

Time Left

40 + 34 =

54 - 25 =

3 × 10 =

6 ÷ 3 =

Next steps

Next, we add a timer to our math quiz and check user answers.

Add a countdown timer

To keep track of time during the quiz, you use a **Timer** component. You also need a variable to store the amount of time that's left.

- ❖ Add an integer variable that's named **timeLeft** in the same way that you declared variables in previous tutorials. Put the **timeLeft** declaration right after the other declarations. Your code should look like the following sample.

*Public Class Form1**' Create a Random object called randomizer
' to generate random numbers.**Private randomizer As New Random**' These integer variables store the numbers
' for the addition problem.**Private addend1 As Integer**Private addend2 As Integer**' These integer variables store the numbers
' for the subtraction problem.**Private minuend As Integer**Private subtrahend As Integer**' These integer variables store the numbers
' for the multiplication problem.**Private multiplicand As Integer**Private multiplier As Integer**' These integer variables store the numbers
' for the division problem.**Private dividend As Integer**Private divisor As Integer**' This integer variable keeps track of the
' remaining time.**Private timeLeft As Integer*

- ❖ In **Windows Forms Designer**, move a Timer control from the Components category of the Toolbox to your form. The control appears in the **gray** area at the bottom of the design window.
- ❖ On the form, select the Timer icon that you just added, and set its Interval property to **1000**. Because this interval is in milliseconds, a value of **1000** causes the timer to raise a Tick event every second.

Check the answers

Because the timer raises a Tick event every second, it makes sense to check the elapsed time in a **Tick Event Handler**. It's also practical to check the answers in that event handler. If time has run out, or if the answers are correct, the quiz should end.

Before you write that event handler, add a method called **CheckTheAnswer()** to determine whether the answers to the math problems are correct. This method should be in line with the other methods, such as **StartTheQuiz()**. Your code should look like the following sample.

```
''' <summary>
''' Check the answers to see if the user got everything right.
''' </summary>
''' <returns>True if the answer's correct, false otherwise.</returns>
''' <remarks></remarks>
Public Function CheckTheAnswer() As Boolean

    If addend1 + addend2 = sum.Value AndAlso
       minuend - subtrahend = difference.Value AndAlso
       multiplicand * multiplier = product.Value AndAlso
       dividend / divisor = quotient.Value Then

        Return True
    Else
        Return False
    End If

End Function
```

This method determines the answers to the math problems and compares the results to the values in the **NumericUpDown** controls. In this code:

- ❖ The Visual Basic version uses the Function keyword instead of the usual Sub keyword because this method returns a value.
- ❖ You can't easily enter the multiplication sign (×) or the division sign (÷) by using the keyboard, so C# and Visual Basic accept an asterisk (*) for multiplication and a slash mark (/) for division.
- ❖ In Visual Basic, the equivalent operator is **AndAlso**. You use the logical and operator to check whether more than one condition is true. In this case, if the values are all correct, the method returns a value of true. Otherwise, the method returns a value of false.
- ❖ The if statement uses the Value property of a **NumericUpDown** control to access the control's current value. In the next section, you use the same property to display the correct answer in each control.

Add an event handler to the timer

Now that you have a way to check the answers, you can write the code for the Tick event handler. This code runs every second, after the timer raises a Tick event. This event handler checks the quiz taker's answers by calling **CheckTheAnswer()**. It also checks how much time has elapsed in the quiz.

On the form, double-click the Timer control, or select it and then select Enter. These actions add a Tick event handler. The code editor appears and displays the Tick handler's method.

Add the following statements to the new event handler method.

```
Private Sub Timer1_Tick() Handles Timer1.Tick  
    If CheckTheAnswer() Then  
        ' If CheckTheAnswer() returns true, then the user  
        ' got the answer right. Stop the timer  
        ' and show a MessageBox.  
        Timer1.Stop()  
        MessageBox.Show("You got all of the answers right!", "Congratulations!")  
        startButton.Enabled = True  
    ElseIf timeLeft > 0 Then  
        ' If CheckTheAnswer() returns false, keep counting  
        ' down. Decrease the time left by one second and  
        ' display the new time left by updating the  
        ' Time Left label.  
        timeLeft -= 1  
        timeLabel.Text = timeLeft & " seconds"  
    Else  
        ' If the user ran out of time, stop the timer, show  
        ' a MessageBox, and fill in the answers.  
        Timer1.Stop()  
        timeLabel.Text = "Time's up!"  
        MessageBox.Show("You didn't finish in time.", "Sorry!")  
        sum.Value = addend1 + addend2  
        difference.Value = minuend - subtrahend  
        product.Value = multiplicand * multiplier  
        quotient.Value = dividend / divisor  
        startButton.Enabled = True  
    End If  
End Sub
```

Each second of the quiz, this method runs. The code first checks the value that **CheckTheAnswer()** returns.

If all answers are correct, that value is true, and the quiz ends:

- ❖ The timer stops.
- ❖ A congratulatory message appears.
- ❖ The Enabled property of the **startButton** control is set to true so that the quiz taker can start another quiz.

If CheckTheAnswer() returns false, the code checks the value of timeLeft:

- ❖ If this variable is greater than 0, the timer subtracts 1 from **timeLeft**. It also updates the Text property of the **timeLabel** control to show the quiz taker how many seconds remain.

- ❖ If no time remains, the timer stops and changes the **timeLabel** text to Time's up! A message box announces that the quiz is over, and the answers are revealed. The start button becomes available again.

Start the timer

To start the timer when the quiz starts, add three lines to the end of the **StartTheQuiz()** method, as the following sample shows.

```
''' <summary>
''' Start the quiz by filling in all of the problem
''' values and starting the timer.
''' </summary>
''' <remarks></remarks>
Public Sub StartTheQuiz()

    ' Fill in the addition problem.
    ' Generate two random numbers to add.
    ' Store the values in the variables 'addend1' and 'addend2'.
    addend1 = randomizer.Next(51)
    addend2 = randomizer.Next(51)

    ' Convert the two randomly generated numbers
    ' into strings so that they can be displayed
    ' in the label controls.
    plusLeftLabel.Text = addend1.ToString()
    plusRightLabel.Text = addend2.ToString()

    ' 'sum' is the name of the NumericUpDown control.
    ' This step makes sure its value is zero before
    ' adding any values to it.
    sum.Value = 0

    ' Fill in the subtraction problem.
    minuend = randomizer.Next(1, 101)
    subtrahend = randomizer.Next(1, minuend)
    minusLeftLabel.Text = minuend.ToString()
    minusRightLabel.Text = subtrahend.ToString()
    difference.Value = 0

    ' Fill in the multiplication problem.
    multiplicand = randomizer.Next(2, 11)
    multiplier = randomizer.Next(2, 11)
    timesLeftLabel.Text = multiplicand.ToString()
    timesRightLabel.Text = multiplier.ToString()
    product.Value = 0

    ' Fill in the division problem.
    divisor = randomizer.Next(2, 11)
    Dim temporaryQuotient As Integer = randomizer.Next(2, 11)
```

```
dividend = divisor * temporaryQuotient  
dividedLeftLabel.Text = dividend.ToString()  
dividedRightLabel.Text = divisor.ToString()  
quotient.Value = 0
```

' Start the timer.

```
timeLeft = 30
```

```
timeLabel.Text = "30 seconds"
```

```
Timer1.Start()
```

End Sub

When your quiz starts, this code sets the **timeLeft** variable to 30 and the Text property of the **timeLabel** control to 30 seconds. Then the **Start()** method of the Timer control starts the countdown.

Run your app

- ❖ Save your program and run it.
- ❖ Select Start the quiz. The timer starts to count down. When time runs out, the quiz ends, and the answers appear.

Start another quiz, and provide correct answers to the math problems. When you answer correctly within the time limit, a message box opens, the start button becomes available, and the timer stops.

Math Quiz

Time Left 19 seconds

34 + 35 = 69

96 - 73 = 23

2 × 10 = 20

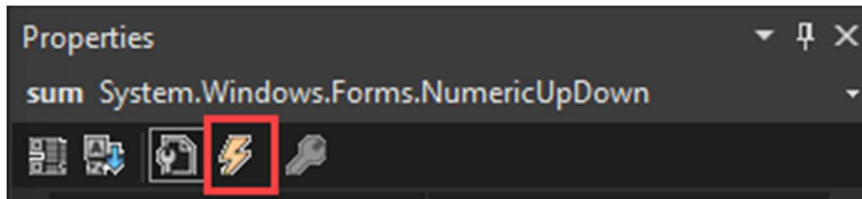
12 ÷ 2 = 6

Start the quiz

Add event handlers for the NumericUpDown controls

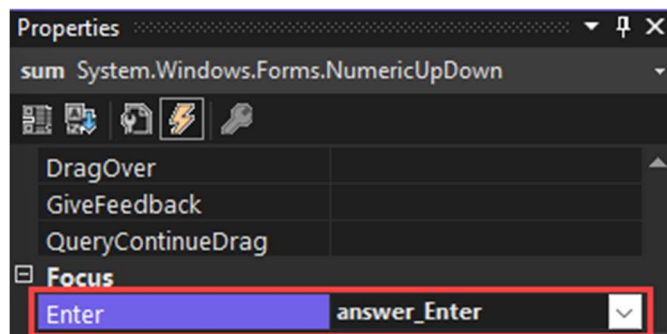
The quiz contains **NumericUpDown** controls that quiz takers use to enter numbers. When you enter an answer, you either need to select the default value first or delete that value manually. By adding an Enter event handler, you can make it easier to enter answers. This code selects and clears the current value in each **NumericUpDown** control as soon as the quiz taker selects it and starts to enter a different value.

- ❖ Select the first **NumericUpDown** control on the form. In the Properties dialog box, select the Events icon on the toolbar.



The Events tab in Properties displays all the events that you can respond to for the item that you selected on the form. In this case, all the listed events pertain to the **NumericUpDown** control.

- ❖ Select the Enter event, enter **answer_Enter**, and then select Enter.



The code editor appears and displays the Enter event handler that you created for the sum **NumericUpDown** control.

- ❖ In the method for the **answer_Enter** event handler, add the following code:

```
''' <summary>
''' Modify the behavior of the NumericUpDown control
''' to make it easier to enter numeric values for
''' the quiz.
''' </summary>
Private Sub answer_Enter(ByVal sender As System.Object, ByVal e As System.EventArgs)
Handles sum.Enter
' Select the whole answer in the NumericUpDown control.
Dim answerBox = TryCast(sender, NumericUpDown)
If answerBox IsNot Nothing Then
    Dim lengthOfAnswer = answerBox.Value.ToString().Length
    answerBox.Select(0, lengthOfAnswer)
End If
End Sub
```

In this code:

- ❖ The first line declares the method. It includes a parameter that's named sender. In Visual Basic, it's **sender As System.Object**. This parameter refers to the object whose event is firing, which is known as the sender. In this case, the sender object is the **NumericUpDown** control.
- ❖ The first line inside the method casts, or converts, the sender from a generic object to a **NumericUpDown** control. That line also assigns the name **answerBox** to the **NumericUpDown** control. All the **NumericUpDown** controls on the form will use this method, not just the addition problem's control.
- ❖ The next line verifies whether **answerBox** was successfully cast as a **NumericUpDown** control.
- ❖ The first line inside the if statement determines the length of the answer that's currently in the **NumericUpDown** control.
- ❖ The second line inside the if statement uses the answer length to select the current value in the control.

When the quiz taker selects the control, Visual Studio fires this event. This code selects the current answer. As soon as the quiz taker starts to enter a different answer, the current answer is cleared and replaced with the new answer.

- ❖ In Windows Forms Designer, select the addition problem's **NumericUpDown** control again.
- ❖ In the Events page of the Properties dialog box, find the Click event, and then select **answer_Enter** from the drop-down menu. This event handler is the one that you just added.
- ❖ In Windows Forms Designer, select the subtraction problem's **NumericUpDown** control.
- ❖ In the Events page of the Properties dialog box, find the Enter event, and then select **answer_Enter** from the drop-down menu. This event handler is the one that you just added. Repeat this step for the Click event.
- ❖ Repeat the previous two steps for the multiplication and division **NumericUpDown** controls.

Run your app

- ❖ Save your program and run it.
- ❖ Start a quiz, and select a **NumericUpDown** control. The existing value is automatically selected and then cleared when you start to enter a different value.

Math Quiz

Time Left 26 seconds

26 + 34 = 0

47 - 26 = 0

3 × 3 = 0

64 ÷ 8 = 0

Start the quiz

Customize our quiz

Here we shall explore some ways to customize the quiz and expand on what we've learned.

Change the color of a label

Use the **BackColor** property of the **timeLabel** control to *turn this label red when only five seconds remain in a quiz.*

```
timeLabel.BackColor = Color.Red
```

Play a sound for a correct answer

Give the quiz taker a hint by playing a sound when the correct answer is entered into a **NumericUpDown** control. To implement this functionality, write an event handler for each control's **ValueChanged** event. This type of event fires whenever a quiz taker changes the control's value.